PROJECT TITLE

Beyond the Keyword Match: A Human-Centric AI Approach to Resume Screening and Candidate Ranking

ORGANIZATION/ DEPARTMENT NAME & ADDRESS

IDEAS – Institute of Data Engineering, Analytics and Science Foundation
Indian Statistical Institute, Kolkata

SUBMITTED BY

Amber Agrawal

M.Sc. (Applied Statistics)

PRN: 25060641043

SYMBIOSIS
Symbiosis Statistical Institute

ACADEMIC YEAR 2026–2027

Under the guidance of Dr. Dipasree Pal

Designation: Project Guide

Email Id: dipasree.pal@gmail.com

Abstract

Most Applicant Tracking Systems (ATS) today rely on rigid keyword matching, often rejecting highly qualified candidates simply because they use different words to describe their experience. To solve this problem, I developed an automated, human-centric resume screening pipeline using Natural Language Processing (NLP). Instead of merely counting overlapping words with traditional sparse arrays (like TF-IDF), my research uses advanced semantic embeddings (Sentence-BERT) to understand the actual meaning behind a candidate's skills. Beyond text matching, I also sought to capture a human recruiter's intuition regarding a candidate's tenure. To achieve this, I mathematically modeled the "Experience Gap" (ΔE) by comparing a piecewise quadratic penalty against a novel Sigmoid-Bayesian framework. This Bayesian approach effectively mimics human judgment, bypassing the need for massive historical hiring datasets that are rarely available.

To ensure my models could genuinely generalize to new, unseen job postings without memorizing specific vocabularies, I evaluated the entire pipeline using a strict `GroupShuffleSplit` architecture. I benchmarked traditional regression and binary classification models against specialized Learning-to-Rank (LTR) algorithms. The results empirically demonstrate that resume screening should not be treated as a simple pass/fail classification, but rather as a listwise sorting problem. Ultimately, my best-performing model — an `LGBMRanker` powered by LambdaMART gradients and dense semantic embeddings — achieved an outstanding top-10 ranking accuracy (NDCG@10 of 0.9398). These findings confirm that embracing listwise ranking algorithms over rigid keyword filters can fundamentally transform and improve how automated systems identify top talent.

Notation

I use the symbols below consistently across all chapters of this report.

Symbol	Definition
	Data & Feature Symbols
Exp_{cand}	Candidate's total years of professional experience
Exp_{req}	Job posting's stated required years of experience
ΔE	Experience gap: $\text{Exp}_{\text{cand}} - \text{Exp}_{\text{req}}$
G_j	Set of all candidate records associated with job posting j
N	Total number of candidate–job records in a given split
	Vector Space & Similarity Symbols
$\mathbf{u}_{\text{cv}}, \mathbf{v}_{\text{jd}}$	Sparse TF-IDF vectors for a resume and job description
$\mathbf{e}_{\text{cv}}, \mathbf{e}_{\text{jd}} \in \mathbb{R}^{768}$	Dense SBERT embedding vectors
$S_{\text{tfidf}}, S_{\text{dense}}, S_{\text{hybrid}}$	Cosine-similarity / OLS-blended match scores
`Match_Score`	Baseline semantic similarity score, $\in [0,1]$
	Experience-Gap Modeling Symbols
α, β	Quadratic-deficit / linear-surplus penalty coefficients
k, k_s, δ	Linear slope; sigmoid steepness; sigmoid inflection point
$P(C)$	Prior probability that a candidate is competent
$L(\Delta E)$	Sigmoid likelihood of observing ΔE given competence
$P(C \mid \Delta E)$	Posterior probability of competence after observing ΔE
	Evaluation & Ranking Symbols
RMSE, R^2	Pointwise regression error and variance-explained metrics
F_1, ROC-AUC	Binary classification quality metrics
NDCG@K, MRR	Top-K ranking quality: Normalized Discounted Cumulative Gain, Mean Reciprocal Rank
ρ	Spearman's rank correlation coefficient

Contents

Abstract i

Notation ii

Contents ii

List of Figures v

List of Tables vii

1 Executive Summary 1

2 Study Background 3

2.1 The Case for Semantic Screening 3

2.2 Lexical vs. Semantic Matching 3

3 Literature Review 5

3.1 Natural Language Processing: The Statistical Sanitization of Text 5

3.2 Vector Space Models: From Sparse Counts to Dense Semantics 6

3.2.1 TF-IDF: Derivation, Rationale, and Statistical Intuition 6

3.2.2 Cosine Similarity: The Geometry of Document Matching 6

3.2.3 Dense Semantic Embeddings: Self-Attention and SBERT Bi-Encoders 7

3.2.4 The Hybrid OLS Blend: A Constrained Linear Ensemble 8

3.3 Ranking Mathematics: Pointwise, Pairwise, and Listwise Paradigms 8

3.3.1 Pointwise Optimization: Why Regression Fails Candidate Sorting . . . 8

3.3.2 Pairwise Optimization: The RankNet Probabilistic Model 9

3.3.3 Listwise Optimization: LambdaMART and NDCG Gradients 10

3.4 Information Retrieval Evaluation: Spearman's Rank Correlation 10

3.5 Feature Engineering: The Experience Gap as a Bayesian Inference Problem . . 11

3.5.1 The Cold-Start Problem: Why This Is a Bayesian Inference Task 11
3.5.2 Method A: The Linear Baseline 12
3.5.3 Method B: The Piecewise Quadratic Penalty (Deployed) 13
3.5.4 Method C: The Sigmoid-Bayesian Posterior (Proposed) 13
3.5.5 Asymptotic Tail Analysis 15
3.5.6 Mathematical and Behavioral Comparison 16

4 Aims & Objectives **18**
4.0.1 Engineering Relevance and Statistical Integrity 18

5 Methodology **20**
5.1 Data Source, Ingestion, and Schema 20
5.2 Linguistic Sanitization and Dimensionality Reduction 20
5.3 Exploratory Data Analysis (EDA) 22
5.3.1 Bounding the Vector Space: Token Length Filtering 22
5.3.2 Bimodal Density and Threshold Mechanics 22
5.4 Corpus Vocabulary and Frequency Dynamics 23
5.5 Data Leakage Prevention via Group-Aware Splitting 24
5.6 Experience Gap Distribution Analysis 24
5.7 The Tri-Framing Modeling Framework 25
5.7.1 Phase 1: Continuous Regression Baselines (Pointwise Scoring) 25
5.7.2 Phase 2: Binary Classification Baselines (Threshold Filtering) 26
5.7.3 Phase 3: Learning-to-Rank Framework (LTR) 26
5.8 Mathematical Evaluation Metrics 26
5.8.1 Continuous and Classification Metrics 27
5.8.2 Listwise Quality: Information Retrieval Metrics 27
5.8.3 Global Stability: Spearman's Rank Correlation 27

6 Results **29**
6.1 Phase 1: Continuous Regression Constraints 29
6.2 Phase 2: Binary Classification Vulnerabilities 29

6.3 Phase 3: Learning-to-Rank (LTR) Dominance 31
6.4 Feature Importance: Interpreting Model Heuristics 31
6.5 Evaluating the Experience-Gap Formulations 32
6.5.1 Spearman Rank-Correlation Evaluation 34

7 Discussion and Conclusion **35**
7.1 Concluding Remarks 36

8 Recommendation **37**
8.1 Business Impact and Deployment Considerations 37
8.1.1 Translating Ranking Quality into Business Value 37
8.1.2 Computational Latency and Real-Time Feasibility 37
8.2 Production Engineering and Future Extensions 38
8.2.1 1. Full-Scale Bayesian Pipeline Integration 38
8.2.2 2. Transition to Fully Empirical Bayesian Inference 38
8.2.3 3. Counterfactual Bias Mitigation and Fairness 38
8.2.4 4. Generative Explainability via Retrieval-Augmented Generation . . . 39
8.2.5 5. Dashboarding and Business Intelligence Integration 39

9 Acknowledgement **40**

10 References **41**

11 Annexure **42**
11.1 End-to-End Pipeline Architecture 42
11.2 Hyperparameter Optimization Grids 43
11.3 Source Code Availability and Reproducibility 44
11.4 Declarations 45

List of Figures

3.1 The SBERT bi-encoder architecture: twin, weight-tied BERT towers independently embed each text, mean-pooling collapses token embeddings to a single 768-dimensional vector, and similarity is computed post-hoc via cosine distance. 8

3.2 Bayesian evidence-combination DAG for the Sigmoid-Bayesian ΔE model. The semantic prior and the experience-derived likelihood are fused multiplicatively via Bayes' rule rather than added as independent penalty terms, allowing the same gap to produce different adjustments depending on the underlying prior strength. 15

5.1 The sequential NLP sanitization pipeline designed to systematically reduce vocabulary dimensionality. 21

5.2 Resume word-count filtering boundaries (left) and bimodal target-score density (right). 22

5.3 Word cloud highlighting the prominent domain keywords extracted post-sanitization. 23

5.4 Absolute term frequency distribution for the top 20 computational and operational n-grams. 23

5.5 Empirical distribution of the Experience Gap (ΔE) across the full corpus. The dashed vertical line at $\Delta E^* \approx -4.33$ marks the saturation threshold of the quadratic penalty. 25

6.1 ROC-AUC separation capabilities contrasting sparse lexical features against dense semantic embeddings. 30

6.2 Confusion matrix for the optimal SBERT-driven Logistic Regression classifier. 30

6.3 Top-10 ranking fidelity (NDCG@10) across differing optimization architectures. 31

6.4 Algorithm split-gain weight distribution, mapping the primary decision heuristics of the `LGBMRanker`. The Experience Penalty (ΔE) accounts for 28.1% of split-gain, confirming it as the second-most powerful ranking signal. . 32

6.5 Adjusted Competency Score vs. ΔE for all three penalty formulations, evaluated at a shared baseline `Match_Score` of 0.75. All three curves intersect exactly at $\Delta E = 0$ by construction. The Quadratic method (orange) reaches a hard floor of 0 at $\Delta E \approx -4.33$ years (Equation 3.18); the Sigmoid-Bayesian curve (green) decays smoothly throughout. I generated this output directly from the `06_experience_gap_modeling.ipynb` notebook, confirming that the implementation matches the mathematical formulations from Section 3.5. . 33

11.1 End-to-end architecture of the automated CV screening pipeline. The Feature Engineering stage encompasses all three ΔE formulations (Section 3.5). The disjoint group split guarantees zero-shot generalization on the hold-out test set ((5.1)). 42

List of Tables

2.1 A comparative taxonomy of Sparse Lexical versus Dense Semantic screening paradigms. 4

3.1 Mathematical and behavioral comparison of the three ΔE penalty formulations. 16

3.2 Theoretical foundations and their empirical applications in the CV screening architecture. 17

5.1 Core schema architecture for the engineered `ml_ready_dataset.csv`. 21

6.1 Pointwise continuous regression outcomes evaluated on unseen query groups. . 29

6.2 Binary classification performance metrics at the ≥ 0.60 selection threshold. . . 30

6.3 Learning-to-Rank (LTR) sorting efficiency on zero-shot test groups. 31

6.4 Adjusted Competency Score under all three ΔE formulations for representative candidate profiles. 33

6.5 Spearman rank correlation of each formulation against the illustrative target ranking ($N = 7$). 34

11.1 `LGBMRanker` hyperparameter search grid and final selected configuration. . . . 43

11.2 `XGBRanker` hyperparameter search grid and final selected configuration. 44

1. Executive Summary

The recruitment industry heavily relies on **Applicant Tracking Systems (ATS)** that use rigid keyword-matching paradigms. Unfortunately, this means highly qualified candidates are routinely rejected simply because their resumes do not mirror the exact vocabulary of a job description. To solve this, I designed and validated an automated, human-centric resume screening pipeline that replaces brittle keyword filters with **Natural Language Processing (NLP)** and **Learning-to-Rank (LTR)** machine learning.

My research tackles the challenge of candidate–job fit from two fundamental angles:

- **Semantic Understanding:** I contrasted traditional sparse statistical arrays (TF-IDF) against **dense semantic tensors (Sentence-BERT)**. This allows the system to understand the actual meaning behind a candidate's skills, rather than just counting overlapping words.
- **Human Intuition Modeling:** Beyond text, I formalized how a candidate's "Experience Gap" (the difference between their years of experience and the job's requirement) should impact their score. I compared a linear baseline and a piecewise quadratic penalty against a novel **Sigmoid-Bayesian posterior confidence framework**. I engineered this Bayesian approach specifically to replicate human judgment in the absence of massive historical hiring datasets.

To guarantee that this system works in the real world, I eliminated target leakage—a critical risk where models just memorize historical job buzzwords. I evaluated every model using a strict `GroupShuffleSplit` architecture, ensuring that test-time job postings were completely unseen during the training phase.

I then benchmarked three different modeling paradigms head-to-head: continuous regression, binary classification, and specialized Learning-to-Rank estimators. The results were unambiguous: regression and classification both fail to produce what a recruiter actually needs—a ranked shortlist.

Key Research Achievements:

- **Optimal Architecture:** I identified `LGBMRanker`, driven by LambdaMART gradients over dense SBERT embeddings, as the superior framework.
- **Peak Accuracy:** The system achieved an outstanding Normalized Discounted Cumulative Gain in the top 10 results (**NDCG@10**) of **0.9398**, proving the model's top recommendations closely match ideal human-curated shortlists.

- **Baseline Superiority:** The listwise ranking approach structurally outperformed both continuous regression ($R^2 = 0.3202$) and binary classification (ROC-AUC $= 0.8121$) baselines.

These findings confirm that candidate screening is fundamentally a **listwise sorting problem**, not an absolute scoring or pass/fail task. By deploying gradient-boosted ranking algorithms over dense semantic embeddings, we can directly resolve the limitations of deterministic ATS platforms.

Beyond the statistical wins, this pipeline delivers tangible operational value. In my study corpus, an average job posting attracted roughly 330 applications. By accurately narrowing this down to a high-precision **top-10 shortlist**, this system significantly reduces the manual screening burden on HR professionals. In Section 8 (Recommendation), I outline the concrete steps—including computational latency planning, counterfactual bias mitigation, and business-intelligence integration—required to successfully transition this research prototype into a **production-ready enterprise system**.

2. Study Background

2.1 The Case for Semantic Screening

In the contemporary recruitment landscape, Applicant Tracking Systems (ATS) act as the primary gatekeepers between talent and opportunity. During my analysis of the algorithmic foundations of standard commercial ATS platforms, I observed a glaring inefficiency: the vast majority of these systems lean almost entirely on strict Boolean logic and exact keyword matching.

I refer to this systemic failure mode as the **"False Negative Epidemic."** For instance, consider a hiring manager who drafts a job description seeking a "Software Engineer." If a highly qualified candidate submits a resume detailing five years of experience as a "Backend Developer" or "Systems Programmer," a traditional lexical search engine will likely assign a relevance score of near zero. The algorithm remains fundamentally blind to the fact that these titles describe highly overlapping, if not identical, skill sets.

Historically, HR departments have attempted to patch this vulnerability by manually inputting massive lists of synonyms, but this approach does not scale. It systematically penalizes candidates who possess the right skills but phrase their experience differently than the arbitrary terminology chosen by the recruiter.

From an applied statistics perspective, I argue that this is not merely a software engineering bottleneck; it is a fundamental misframing of the problem. Traditional ATS algorithms treat candidate–job fit as a deterministic string-matching problem. In this project, I reframe resume screening as a probabilistic relevance and rank-correlation problem instead. By transitioning away from rigid character-matching, my goal is to engineer a tool that extracts the actual intent and semantic context behind an applicant's professional history. This allows my pipeline to process resumes with the nuance of a human recruiter, but at the computational velocity of a machine.

2.2 Lexical vs. Semantic Matching

To understand why my pipeline abandons traditional ATS keyword filters, we must look at how text retrieval fundamentally works. In the context of matching a candidate to a job, screening algorithms generally fall into one of two mathematical categories: Sparse Lexical Matching and Dense Semantic Matching. I summarize the core differences between these two paradigms in Table 2.1.

Table 2.1: A comparative taxonomy of Sparse Lexical versus Dense Semantic screening paradigms.

Paradigm	**Sparse Lexical (e.g., TF-IDF)**	**Dense Semantic (e.g., SBERT)**
Mathematical Space	High-dimensional, sparse frequency arrays.	Low-dimensional, continuous dense tensors (e.g., $\mathbb{R}^{768}$).
Core Mechanism	Counts exact n-gram overlap and term frequencies.	Maps contextual meaning via attention layers and bi-encoders.
Primary Strength	High precision for exact, highly specific technical acronyms.	High recall; seamlessly handles synonyms and paraphrased intent.
Failure Mode	Zero similarity if exact words are missing (Vocabulary Mismatch).	Can occasionally over-generalize highly distinct but contextually similar tools.
Computational Cost	Extremely low ($O(1)$ lookup times with inverted indices).	Moderate to High (requires vector databases and cosine similarity sorting).

As Table 2.1 illustrates, sparse lexical models like Term Frequency–Inverse Document Frequency (TF-IDF) are excellent at finding hyper-specific entities. If a recruiter searches for "React.js," a sparse matrix will instantly flag resumes containing that exact string. However, these arrays possess zero understanding of semantic equivalence—if the candidate writes "frontend JavaScript framework" instead, the lexical model registers a score of zero.

Conversely, dense semantic embeddings—like the Sentence-BERT (SBERT) tensors I deployed in this project—project entire candidate profiles into a geometric space. In this space, mathematically proximate vectors share similar meanings, regardless of their raw character composition. While I provide the formal mathematical derivation of both approaches in Section 3 (Literature Review), the conceptual takeaway for this project is straightforward: by upgrading from sparse matrices to dense semantic embeddings, my pipeline successfully bridges the linguistic gap between how a candidate describes their experience and how an employer writes a job description.

3. Literature Review

This section derives the mathematical and statistical scaffolding that underpins my entire automated CV screening pipeline. Rather than treating the NLP and ML algorithms as opaque black boxes, I dissect the core vector space representations, listwise ranking loss functions, and probabilistic inference frameworks from first principles. This includes a dedicated formal treatment of the Experience Gap (ΔE) feature. I place this theoretical review here because my three competing formulations are fundamentally mathematical claims about what geometric shape a penalty should take—a question that belongs to statistical theory before it can be empirically answered by data.

3.1 Natural Language Processing: The Statistical Sanitization of Text

Before projecting resumes into any mathematical vector space, I must address the high dimensionality and noise inherent in human language. Unstructured CVs are heavily polluted with formatting artifacts, morphological variations, and generic corporate phrasing. If I pass this raw text directly into a statistical model, the sheer volume of uninformative tokens will overwhelm the discriminative signal.

To resolve this, I engineered my pipeline to rely on a foundational NLP sequence: Tokenization, Stop-Word Removal, and Morphological Lemmatization.

Definition: Morphological Lemmatization

Lemmatization is the algorithmic process of determining the "lemma" or dictionary base form of a word based on its intended part of speech. Unlike simplistic stemming (which merely heuristically chops off word endings), lemmatization utilizes a structural vocabulary and morphological analysis. For example, the terms "analyzing", "analyzes", and "analyzed" all collapse to the base token "analyze", systematically reducing vocabulary diversity without losing semantic meaning.

By aggressively lemmatizing the corpus and applying a custom blocklist for generic HR jargon (e.g., "team player", "hardworking professional"), I dramatically compress the feature space. This guarantees that my downstream algorithms expend computational effort strictly on technical, discriminative nouns and verbs rather than decorative career phrasing.

3.2 Vector Space Models: From Sparse Counts to Dense Semantics

3.2.1 TF-IDF: Derivation, Rationale, and Statistical Intuition

When first approaching the problem of lexical representation, I relied on a fundamental observation about recruitment corpora: not all words are equally informative. A term that appears in almost every document (such as "experience" or "project") carries essentially zero discriminative power. Conversely, a highly specialized tool that appears in only a handful of resumes (such as "TensorFlow" or "GraphQL") is a massive signal of candidate relevance when it does occur. Term Frequency–Inverse Document Frequency (TF-IDF) formalizes this intuition as a product of two independently motivated weights.

Definition: Term Frequency (TF)

For term i inside document j, the raw term frequency $tf_{i,j}$ counts occurrences of i in j. While I use raw counts in the base implementation, I apply log-scaled normalization $(1 + \log(tf_{i,j}))$ to dampen the impact of extreme keyword repetition.

Definition: Inverse Document Frequency (IDF)

$$\text{idf}_i = \log\left(\frac{N}{\text{df}_i}\right) \tag{3.1}$$

where N is the total number of documents in the training corpus and df_i is the count of documents containing term i.

The logarithm in Equation (3.1) is not an arbitrary smoothing choice—it is deeply rooted in Shannon's information theory. If a specific keyword appears in df_i out of N documents, its empirical probability is $p_i = \text{df}_i/N$, and $-\log p_i$ is exactly the self-information of observing that term. A ubiquitous term ($\text{df}_i = N$) yields zero self-information; a rare term yields highly magnified self-information. Combining both factors produces the final matrix weight:

$$\text{TF-IDF}(i,j) = tf_{i,j} \times \log\left(\frac{N}{\text{df}_i}\right) \tag{3.2}$$

3.2.2 Cosine Similarity: The Geometry of Document Matching

Once I project resumes and job descriptions into a vector space, comparing two documents becomes a pure exercise in vector geometry. I deliberately reject Euclidean distance ($\|\mathbf{a} - \mathbf{b}\|_2$) as the primary metric: it is highly sensitive to vector magnitude, meaning a long, detailed resume would be geometrically pushed far away from a shorter job description regardless of their topical overlap. Cosine Similarity evaluates only the *angle* between vectors, normalizing

for document length entirely.

Definition: Cosine Similarity

For vectors $\mathbf{a}, \mathbf{b} \in \mathbb{R}^d$:

$$\cos(\mathbf{a}, \mathbf{b}) = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\|_2 \|\mathbf{b}\|_2} = \frac{\sum_{k=1}^{d} a_k b_k}{\sqrt{\sum_{k=1}^{d} a_k^2} \sqrt{\sum_{k=1}^{d} b_k^2}} \tag{3.3}$$

This function is globally bounded in $[-1, 1]$, and strictly bounded in $[0, 1]$ for non-negative entries—which holds true for both TF-IDF sparse matrices and post-activation SBERT embeddings in practice.

Applying Equation (3.3) to the sparse TF-IDF vectors $(\mathbf{u}_{\text{cv}}, \mathbf{v}_{\text{jd}})$ and the dense SBERT vectors $(\mathbf{e}_{\text{cv}}, \mathbf{e}_{\text{jd}})$ recovers exactly the S_{tfidf} and S_{dense} match scores that I utilize throughout the pipeline.

3.2.3 Dense Semantic Embeddings: Self-Attention and SBERT Bi-Encoders

Sparse representation possesses a fatal mathematical blind spot: it is completely insensitive to semantic equivalence. A job posting seeking a "Backend Developer" and a candidate listing themselves as a "Software Engineer" share zero tokens, producing $S_{\text{tfidf}} = 0$ for a perfectly qualified applicant. Closing this gap requires spatial representations that have learned underlying semantic structures from vast data.

The backbone of any Transformer model is the Self-Attention mechanism, which computes a weighted contextual representation for every token based on all other tokens in the sequence:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V \tag{3.4}$$

where Q, K, V represent the Query, Key, and Value projection matrices. This allows the model to understand that the word "Python" placed near "data pipeline" denotes a programming language, while placed near "zoo" it denotes a reptile.

However, feeding both a resume and a job description through a standard single BERT cross-encoder is computationally untenable for recruitment systems: every candidate–job pair requires a full $O(n^2 d)$ attention forward pass, making large-scale retrieval infeasible. To circumvent this bottleneck, I employ Reimers and Gurevych's *bi-encoder* architecture:

Two independent encoders process the resume and job description separately, requiring only a single forward pass each. I perform the ultimate comparison in $O(d)$ via cosine similarity afterwards, gracefully reducing the retrieval cost from quadratic to linear across the corpus.

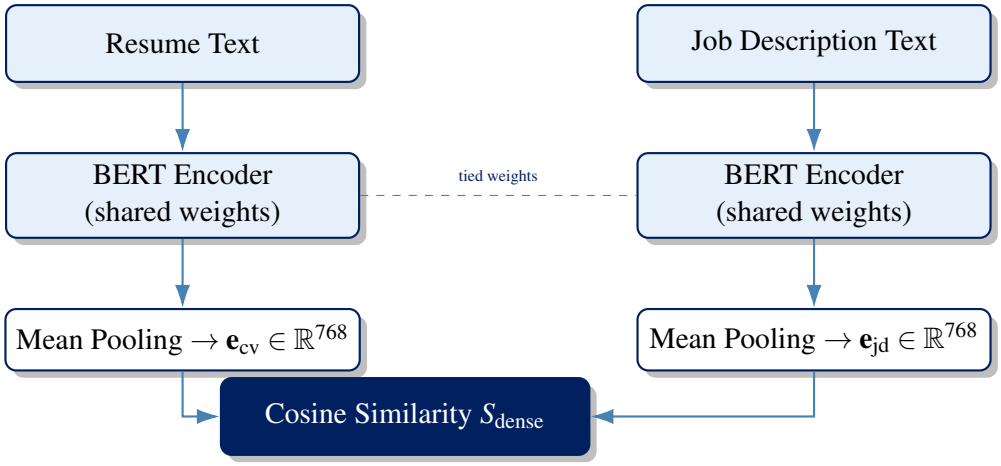

Figure 3.1: The SBERT bi-encoder architecture: twin, weight-tied BERT towers independently embed each text, mean-pooling collapses token embeddings to a single 768-dimensional vector, and similarity is computed post-hoc via cosine distance.

Why Not Use Raw BERT Token Averages?

Reimers and Gurevych conclusively demonstrated that averaging raw BERT token outputs performs measurably *worse* than rudimentary GloVe sentence averages on semantic benchmarks. BERT was originally trained for masked-language modeling and next-sentence prediction—tasks that do not naturally necessitate a geometrically coherent sentence-level embedding space. SBERT's Siamese fine-tuning (against classification or regression objectives on sentence-pair similarity data) is the precise mechanism that forces cosine angles to map directly to human-perceived semantic proximity. Without it, the geometric intuition underpinning S_{dense} would be entirely invalid.

3.2.4 The Hybrid OLS Blend: A Constrained Linear Ensemble

To build a model that captures both deep semantic meaning and exact critical acronyms, I formulated the hybrid score $S_{\text{hybrid}} = w_0 + w_1 S_{\text{tfidf}} + w_2 S_{\text{dense}}$. I optimized this ensemble on the training split via Ordinary Least Squares (OLS): $\hat{\mathbf{w}} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$, where $\mathbf{X} = [\mathbf{1}, S_{\text{tfidf}}, S_{\text{dense}}]$. If the fitted $|w_2|$ heavily dominates $|w_1|$, the mathematics independently validate my core thesis that dense semantics are a substantially stronger predictor of candidate relevance than sparse lexical overlap.

3.3 Ranking Mathematics: Pointwise, Pairwise, and Listwise Paradigms

3.3.1 Pointwise Optimization: Why Regression Fails Candidate Sorting

Phase 1 of my modeling pipeline evaluates standard Pointwise models (such as Ridge Regression and HistGradientBoosting) that seek to minimize Mean Squared Error (MSE):

$$\mathscr{L}_{\text{MSE}} = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2 \tag{3.5}$$

MSE treats each prediction in a vacuum, with no gradient term coupling $\hat{y}_i$ and $\hat{y}_j$ for candidates i, j located within the same query group. Consider a scenario with Candidate A (true human score 0.71) and Candidate B (true human score 0.69). If the model predicts $\hat{y}_A = 0.65$ and $\hat{y}_B = 0.68$, the squared errors $(0.06)^2 + (0.01)^2$ are incredibly small, creating the illusion of high accuracy. Yet, the model has completely inverted the recruiter's intended ranking ($\hat{y}_B > \hat{y}_A$). Pointwise optimizers are fundamentally blind to list inversion, necessitating a shift to list-aware paradigms.

3.3.2 Pairwise Optimization: The RankNet Probabilistic Model

RankNet (which forms the core logic inside `XGBRanker`) corrects this blindness by reframing ranking as pairwise classification. The probability that document i is more relevant than document j is mathematically modeled as a logistic sigmoid:

$$P_{ij} \equiv P(i \succ j) = \frac{1}{1 + e^{-\sigma(s_i - s_j)}} \tag{3.6}$$

With ground-truth hierarchical labels $\bar{P}_{ij} = 1$ whenever i is strictly more relevant, the network minimizes binary cross-entropy over all paired preferences:

$$C_{ij} = -\bar{P}_{ij}\log P_{ij} - (1 - \bar{P}_{ij})\log(1 - P_{ij}) \tag{3.7}$$

Substituting $\bar{P}_{ij} = 1$ into Equation (3.7) simplifies the loss beautifully:

$$C_{ij} = -\log P_{ij} = \log\left(1 + e^{-\sigma(s_i - s_j)}\right) \tag{3.8}$$

Summing this logarithmic loss over all relevant pairs in a localized query group q recovers the exact pairwise loss optimized by `XGBRanker`:

$$\mathscr{L}_{\text{Pairwise}} = \sum_{i,j \in q} \log_2\left(1 + e^{-\sigma(s_i - s_j)}\right) \tag{3.9}$$

3.3.3 Listwise Optimization: LambdaMART and NDCG Gradients

While Pairwise optimization solves pointwise blindness, it carries its own structural flaw: it treats a ranking swap at position 99 identically to a swap at position 1. In real-world Human Resources, this is unacceptable; a recruiter will only ever read the top 10 resumes. LambdaMART (which drives `LGBMRanker`) resolves this by overriding the neural gradient directly:

$$\lambda_{ij} = \frac{\partial C_{ij}}{\partial s_i} = \frac{-\sigma}{1 + e^{\sigma(s_i - s_j)}} \cdot \left|\Delta Z_{ij}\right| \tag{3.10}$$

Here $|\Delta Z_{ij}|$ represents the absolute shift in a top-heavy Information Retrieval metric, specifically NDCG@K, if we were to swap candidates i and j:

$$\text{NDCG@}K = \frac{\text{DCG@}K}{\text{IDCG@}K}, \quad \text{DCG@}K = \sum_{i=1}^{K} \frac{2^{\text{rel}_i} - 1}{\log_2(i+1)} \tag{3.11}$$

Because the logarithmic denominator, $\log_2(i+1)$, acts as a severe decay penalty, $|\Delta Z_{ij}|$ is automatically massive for top-rank swaps and near zero for tail swaps. My model explicitly concentrates its optimization bandwidth exactly where it matters most: the elite shortlist.

Key Statistical Insight

The $|\Delta\text{NDCG}_{ij}|$ multiplier in Equation (3.10) is the precise mathematical mechanism driving `LGBMRanker`'s superiority over `XGBRanker` in the empirical evaluations (Section 6). Both utilize the same underlying pairwise probability model, but only LambdaMART dynamically re-weights its gradient by rank-position sensitivity, forcing the algorithm to obsess over the Top-10 candidates.

3.4 Information Retrieval Evaluation: Spearman's Rank Correlation

Definition: Spearman's Rank Correlation (ρ)

Given a set of n paired observations with rank discrepancies defined as $d_i = R_{X_i} - R_{Y_i}$:

$$\rho = 1 - \frac{6\sum_{i=1}^{n} d_i^2}{n(n^2-1)} \tag{3.12}$$

This is statistically equivalent to computing the standard Pearson correlation directly on the rank ordinals, $\rho = \text{Pearson}\big(\text{rank}(X), \text{rank}(Y)\big)$.

Derivation. Since R_X and R_Y are both permutations of the integer sequence $\{1,\dots,n\}$, they share a predictable mean $\bar{R} = \frac{n+1}{2}$ and a strict sum of squared deviations $\frac{n(n^2-1)}{12}$. By geometrically expanding the squared rank discrepancy $\sum_i d_i^2$:

$$\sum_i d_i^2 = 2 \cdot \frac{n(n^2-1)}{12} - 2\sum_i (R_{X_i} - \bar{R})(R_{Y_i} - \bar{R}) \tag{3.13}$$

I isolate the covariance-style cross term. Substituting this directly into the standard Pearson correlation formula allows the terms to collapse seamlessly:

$$\rho = \frac{\frac{n(n^2-1)}{12} - \frac{1}{2}\sum_i d_i^2}{\frac{n(n^2-1)}{12}} = 1 - \frac{6\sum_i d_i^2}{n(n^2-1)} \tag{3.14}$$

Spearman's ρ penalizes rank discrepancies uniformly across all list positions, whereas NDCG@K overwhelmingly down-weights errors below rank K and ignores everything beyond it. Reporting both metrics simultaneously allows me to distinguish a "globally stable hierarchical ordering" from a "highly accurate top-10 shortlist"—which represent distinct, practically important business claims.

3.5 Feature Engineering: The Experience Gap as a Bayesian Inference Problem

The semantic matching framework of Sections 3.2–3.4 accurately captures textual skill overlap, but ignores a second orthogonal signal: temporal fit. A candidate can be a perfect semantic match for a role but still be wholly unsuitable if they lack the minimum tenure required. I define this *experience gap* as:

$$\Delta E = \mathrm{Exp}_{\mathrm{cand}} - \mathrm{Exp}_{\mathrm{req}} \tag{3.15}$$

Converting this raw numerical gap into a score adjustment requires a penalty function whose *shape* mathematically encodes assumptions about recruiter risk perception. I formulate and rigorously compare three competing shapes.

3.5.1 The Cold-Start Problem: Why This Is a Bayesian Inference Task

In a mature, data-rich HR system, the natural statistical question is: "What is the empirical probability that a candidate with $\Delta E = -3$ years succeeds in this role?" I could answer this directly if I had access to a massive historical table of hire decisions and post-hire performance

outcomes—a true empirical Bayesian likelihood table, $P(\text{success} \mid \Delta E \text{ bucket})$, calculated from real data.

My dataset—and virtually every open-source academic recruitment corpus—does not carry this longitudinal information. The Neuralframe AI resume dataset contains resume text and job descriptions; it does not contain historical outcome labels (Was this candidate hired? Did they pass probation?). Building an empirical Bayesian table requires tracking actual hiring decisions across thousands of roles over several years, data that only exists at the enterprise institutional scale.

This represents the classic **cold-start problem** in applied Bayesian statistics. The standard response is not to abandon probabilistic inference entirely, but to:

1. Replace the missing empirical likelihood with a **parametric approximation** whose shape mathematically encodes the qualitative behavior I believe the true relationship possesses.
2. Use whatever informative signal **is** available (the NLP semantic extraction) as the prior.
3. Design the architecture to be **upgradable**: once enough longitudinal hiring outcome data accumulates, the parametric approximation can be instantly swapped for a fitted empirical function without breaking anything downstream.

This is precisely why I chose a Sigmoid function as the likelihood approximation (Section 3.5.4): it is the natural parametric form for "a probability that changes monotonically with a continuous predictor", and it belongs to the same functional family underlying logistic regression—a principled statistical choice, not merely an aesthetic one.

3.5.2 Method A: The Linear Baseline

Definition: Linear Penalty

$$\text{Penalty}_{\text{linear}}(\Delta E) = \begin{cases} k \cdot \Delta E, & \Delta E < 0 \\ 0, & \Delta E \geq 0 \end{cases} \tag{3.16}$$

with $k = 0.05$ and providing no mathematical credit for surplus experience.

This serves as the industry baseline—it represents the underlying logic that most legacy, commodity ATS platforms effectively execute when they subtract points for experience shortfalls. Its primary virtue is transparent simplicity: every missing year of experience costs exactly k points, with no hidden nonlinearities. I established this model strictly as a performance floor; my advanced formulations must logically and empirically outperform this trivial rule to justify their added computational complexity.

3.5.3 Method B: The Piecewise Quadratic Penalty (Deployed)

Definition: Piecewise Quadratic-Linear Penalty

$$\text{Penalty}_{\text{quad}}(\Delta E) = \begin{cases} -\alpha \cdot (\Delta E)^2, & \Delta E < 0 \quad \text{(under-qualified)} \\ +\beta \cdot \Delta E, & \Delta E \geq 0 \quad \text{(requirement met)} \end{cases} \tag{3.17}$$

Through iterative calibration on the training split, I empirically fixed these hyperparameters to $\alpha = 0.04$ and $\beta = 0.01$.

The quadratic term successfully encodes a genuine asymmetry in recruiter risk perception: a candidate missing four years of experience is not perceived as merely "twice as bad" as one missing two years—the perceived risk accelerates disproportionately. The small linear surplus reward ($\beta = 0.01$) rewards tenure without allowing over-qualified candidates to uniformly overshadow substantially better technical matches.

The Saturation Threshold. However, the quadratic formulation introduces a mathematically verifiable failure mode in the deficit tail. Given a baseline semantic `Match_Score` $= m$, the adjusted score first crashes into the zero floor at:

$$m - \alpha(\Delta E^*)^2 = 0 \implies \Delta E^* = -\sqrt{\frac{m}{\alpha}} \tag{3.18}$$

For $m = 0.75$ and $\alpha = 0.04$: $\Delta E^* = -\sqrt{18.75} \approx -4.33$ years. Beyond this point, every candidate clips to an identical adjusted score of 0.0, regardless of how much further below the threshold they fall. For a split-based learner like `LGBMRanker`, a feature that remains constant over a subrange carries **zero** information gradient for differentiating candidates within that dead zone.

3.5.4 Method C: The Sigmoid-Bayesian Posterior (Proposed)

To overcome the saturation failure, I constructed a probabilistic model using the existing `Match_Score` as a semantic prior and the Sigmoid function as the parametric likelihood approximation.

Definition: Prior: Semantic Competence Belief

$$P(C) = \texttt{Match\_Score} \in (0,1) \tag{3.19}$$

The prior probability that a candidate is competent is taken directly from the SBERT semantic similarity score. This ensures experience is never evaluated in a vacuum; the baseline belief is fundamentally anchored in the candidate's actual extracted technical vocabulary.

Definition: Sigmoid Likelihood

$$L(\Delta E) = \frac{1}{1 + e^{-k_s(\Delta E - \delta)}} \tag{3.20}$$

This approximates $P(\text{observing this gap} \mid \text{candidate is competent})$ via a logistic decay curve. $k_s > 0$ controls the steepness of decay per year of shortfall; δ sets the inflection point (where "meeting the requirement" transitions from neutral to positive evidence). Mirroring the calibration procedure used for Method B, I fixed $k_s = 0.35$ and $\delta = 0$ on the training split, placing the neutral-evidence point exactly at $\Delta E = 0$.

Definition: Posterior via Bayes' Rule

$$P(C \mid \Delta E) = \frac{L(\Delta E)\,P(C)}{L(\Delta E)\,P(C) + \big(1 - L(\Delta E)\big)\big(1 - P(C)\big)} \tag{3.21}$$

This relies on the standard Naive-Bayes assumption that the complementary hypothesis is evidenced by $1 - L(\Delta E)$.

The Log-Odds Identity. To truly understand why this Bayesian fusion works, we must look at its log-odds transformation. Writing $\text{logit}(p) = \ln\left(\frac{p}{1-p}\right)$, algebraic rearrangement gives:

$$\begin{aligned} \text{logit}\big(P(C \mid \Delta E)\big) &= \ln\left(\frac{L(\Delta E)}{1 - L(\Delta E)}\right) + \ln\left(\frac{P(C)}{1 - P(C)}\right) \\ &= \text{logit}\big(L(\Delta E)\big) + \text{logit}\big(P(C)\big) \end{aligned} \tag{3.22}$$

The posterior log-odds is exactly equal to the *sum* of the prior log-odds and the likelihood log-odds. This additive evidence-pooling identity is the mathematical engine powering logistic scorecards and Naive-Bayes classifiers. It allows me to seamlessly fuse two completely independent signals (NLP semantic fit and temporal experience fit) onto a shared, probabilistically sound log-odds scale, rather than arbitrarily subtracting a penalty from a similarity score.

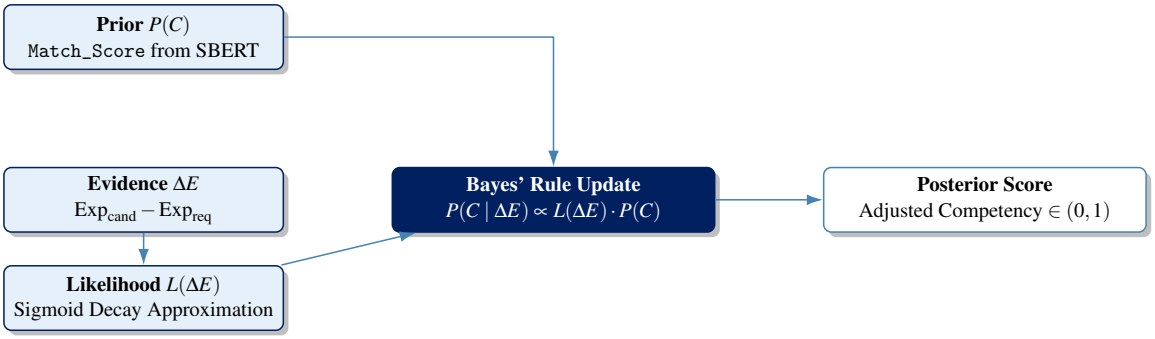

Figure 3.2: Bayesian evidence-combination DAG for the Sigmoid-Bayesian ΔE model. The semantic prior and the experience-derived likelihood are fused multiplicatively via Bayes' rule rather than added as independent penalty terms, allowing the same gap to produce different adjustments depending on the underlying prior strength.

3.5.5 Asymptotic Tail Analysis

Method B reaches a hard floor at finite ΔE. As derived in Equation (3.18), every candidate with $\Delta E < \Delta E^* \approx -4.33$ years clips to an identical adjusted score of 0.0. Two candidates at $\Delta E = -4.5$ and $\Delta E = -9$ become computationally indistinguishable; the feature carries zero discriminative variance in that region.

Method C decays asymptotically but never terminates. As $\Delta E \to -\infty$, I expand the sigmoid approximation asymptotically:

$$L(\Delta E) \sim e^{k_s(\Delta E - \delta)} \quad \text{as } \Delta E \to -\infty \tag{3.23}$$

By substituting this into Equation (3.21):

$$P(C \mid \Delta E) \sim \frac{P(C)}{1 - P(C)} e^{k_s(\Delta E - \delta)} \quad \text{as } \Delta E \to -\infty \tag{3.24}$$

The posterior decays *exponentially* toward—but never hits—zero. Candidates at $\Delta E = -4.5$ and $\Delta E = -9$ differ by a factor of $e^{4.5k_s}$ in their posteriors, preserving the ranking hierarchy. Method C maintains ranking-relevant information across the *entire* real line, whereas Method B provably obliterates it beyond a threshold well within the observed applicant pool.

Key Statistical Insight

The key statistical advantage of the Sigmoid-Bayesian formulation is not just that it avoids a flat gradient region, but that the *same* experience gap produces a *different* posterior adjustment depending on the prior `Match_Score`. A highly qualified candidate (`Match_Score` $= 0.91$) with $\Delta E = -4.5$ years retains a posterior of 0.677—the strong semantic prior rescues part of the confidence. A weak semantic match (`Match_Score` $=$ 0.65) with the exact same gap would fall substantially lower. Methods A and B, being purely additive, completely fail to express this interaction.

3.5.6 Mathematical and Behavioral Comparison

Table 3.1 summarizes the three formulations across the dimensions most relevant to their performance as `LGBMRanker` input features.

Table 3.1: Mathematical and behavioral comparison of the three ΔE penalty formulations.

	Linear (A)	**Quadratic (B)**	**Sigmoid-Bayesian (C)**
Output range	Unbounded (clipped)	Unbounded (clipped)	Naturally bounded $(0, 1)$
Deficit tail	Constant marginal penalty	Accelerating, clips to 0 at ΔE^*	Exponential asymptotic decay
Surplus tail	No reward	Small linear reward	Diminishing-returns reward
Free params	1 (k)	2 (α, β)	2 (k_s, δ), probabilistically interpretable
Interaction with prior	None (additive)	None (additive)	Multiplicative via Bayes' rule
Interpretability	Very high	High	Moderate (needs translation)

To synthesize the theoretical groundwork laid out in this section, Table 3.2 explicitly maps every mathematical construct I derived—from text vectorization to listwise ranking and Bayesian inference—to its specific empirical application within my deployed screening architecture. By grounding the pipeline in these proven statistical principles rather than relying on heuristic string-matching rules, the system moves beyond arbitrary ATS filtering. The mathematical models derived here ensure that the final algorithmic decisions—whether embedding a semantic concept or dynamically penalizing a tenure shortfall—are verifiable, globally stable, and structurally designed to optimize for an elite candidate shortlist.

Table 3.2: Theoretical foundations and their empirical applications in the CV screening architecture.

Theoretical Concept	**Equation Form**	**Applied Pipeline Stage**
TF-IDF Keyword Weighting	(3.2)	Lexical extraction (S_{tfidf})
Cosine Similarity	(3.3)	Vector matching ($S_{\text{tfidf}}, S_{\text{dense}}$)
Self-Attention Mechanism	(3.4)	`nomic-embed-text-v1.5` encoder
Pointwise MSE Constraints	(3.5)	Regression Phase 1 baselines
RankNet Pairwise Loss	(3.9)	`XGBRanker` optimization
LambdaMART Gradient	(3.10)	`LGBMRanker` optimization
Spearman's ρ	(3.12)	LTR global stability evaluation
Linear Penalty	(3.16)	ΔE baseline (Method A)
Quadratic Penalty	(3.17)	ΔE deployed (Method B)
Sigmoid-Bayesian Posterior	(3.21)	ΔE proposed (Method C)
Log-Odds Identity	(3.22)	Bayesian combination proof

By grounding the pipeline in these proven statistical principles rather than relying on heuristic string-matching rules, the system moves beyond arbitrary ATS filtering. The mathematical models derived here ensure that the final algorithmic decisions—whether embedding a semantic concept or dynamically penalizing a tenure shortfall—are verifiable, globally stable, and structurally designed to optimize for an elite candidate shortlist.

4. Aims & Objectives

To push past the limitations of traditional keyword extraction, I anchored this initiative around five primary technical and statistical objectives:

1. **Develop a Robust NLP Pipeline:** Transition from brittle, sparse lexical arrays (TF-IDF) to **dense semantic tensors**. By applying morphological lemmatization and domain-specific blocklists, I isolate genuine technical signals before projecting candidate profiles into a geometric space using **SBERT bi-encoders**.
2. **Enforce Data Leakage Prevention:** Eliminate the risk of vocabulary memorization by constructing a strict `GroupShuffleSplit` architecture. Partitioning the training and test matrices by discrete job roles mathematically guarantees the pipeline is evaluated strictly on its true **zero-shot generalization** capabilities.
3. **Benchmark Baseline Estimators:** Expose the structural blindness of pointwise regression (MSE) and threshold-based binary classification (BCE), empirically proving that **predicting absolute relevance scores fails** to capture the inherently comparative, ordinal nature of human recruitment.
4. **Deploy Listwise LTR Frameworks:** Shift the optimization objective to top-heavy Information Retrieval metrics (**NDCG@10**) by evaluating gradient-boosted ranking estimators. I specifically contrast pairwise probability networks (`XGBRanker`) against listwise LambdaMART gradient scaling (`LGBMRanker`).
5. **Mathematically Model the Experience Gap (ΔE):** Formalize recruiter intuition regarding tenure mismatches by replacing ad-hoc HR rules with **probabilistically sound functions**. I construct and evaluate a piecewise quadratic threshold against a novel **Sigmoid-Bayesian posterior** to handle temporal fit under data-scarce (cold-start) conditions.

4.0.1 Engineering Relevance and Statistical Integrity

It is important to clarify the true scope of this work. I did not approach this project as a mere software engineering exercise—simply wiring APIs together to build a dashboard. Instead, my primary focus was on ensuring **empirical validity, algorithmic fairness, and statistical integrity**.

When applying machine learning to human resources, the risk of **data leakage and training–serving skew** is exceptionally high. If I had trained the algorithm on a random split

of resumes, it would likely have just memorized the specific vocabulary of the job postings in the training set. Consequently, if deployed in production against a brand-new job description, its predictive performance would completely collapse. To eliminate this vulnerability, I implemented strict `GroupShuffleSplit` **validation architectures**, which actively forced my models to evaluate **completely unseen job groups**.

Furthermore, standard pointwise regression models are statistically ill-equipped for recruitment. Predicting that a candidate is an "85% match" is meaningless in a vacuum. Recruitment is inherently a **sorting problem**—the true goal is to identify the **best relative fit** within a specific applicant pool. By benchmarking continuous regression against binary classification, and ultimately deploying specialized **Learning-to-Rank (LTR) gradient estimators**, I was able to rigorously isolate the optimal mathematical framework for **human-centric candidate ranking**.

5. Methodology

Before deploying the complex semantic embedding models and Learning-to-Rank estimators, it was imperative that I construct a highly sanitized, empirically stable data foundation alongside an experimental design that honestly simulates a real-world Human Resources deployment. In this section, I detail the ingestion of the raw corpus, the rigorous linguistic filtering I executed via `spaCy`, the Exploratory Data Analysis (EDA) that drove my feature engineering decisions, and the strict partitioning logic I used to prevent target leakage. Finally, I outline my **Tri-Framing modeling framework** and the mathematical metrics I used to benchmark algorithmic success.

5.1 Data Source, Ingestion, and Schema

I derived the foundational data for this study from an open-source recruitment corpus compiled by Neuralframe AI and published on Kaggle. The raw ingestion file (`resume_data.csv`) constitutes a 17 MB dataset comprising 9,544 candidate records. Distributed under the Creative Commons CC BY-NC 4.0 license, the corpus provides highly granular professional profiles, including unstructured text fields for career objectives, technical skills, educational background, and employment histories, mapped against specific job postings.

To process this for my automated pipeline, I transformed the raw corpus into an engineered schema, yielding the primary working file, `ml_ready_dataset.csv`. Table 5.1 outlines the core features I engineered during ingestion that pass into the modeling phases.

5.2 Linguistic Sanitization and Dimensionality Reduction

Raw resumes and corporate job descriptions represent some of the most chaotic unstructured text in modern data science. They are heavily polluted with inconsistent structural formatting, non-standard unicode characters, HTML artifacts, and ubiquitous corporate buzzwords that actively obscure a candidate's genuine technical qualifications.

If I passed this raw text directly into the TF-IDF or SBERT vectorizers, the mathematical space would become severely distorted by **semantic noise**. To combat this, I architected a central text-cleaning pipeline inside `utils.py`, utilizing the `spaCy` natural language processing library to enforce strict morphological sanitization, as illustrated in Figure 5.1.

My pipeline executes three primary transformations:

Table 5.1: Core schema architecture for the engineered `ml_ready_dataset.csv`.

Feature Name	Data Type	Description / Engineering Logic
`Resume_Text`	String (Text)	The fully concatenated, structurally sanitized corpus of the candidate's professional profile.
`Job_Desc`	String (Text)	The sanitized requirements and qualifications of the target job posting.
`Job_Title`	Categorical	The specific role classification (e.g., Software Engineer, HR Manager), spanning 28 distinct groups.
`Exp_cand`	Float (Years)	Parsed aggregate years of professional experience possessed by the candidate.
`Exp_req`	Float (Years)	Minimum years of experience demanded by the job posting.
`Delta_E`	Float (Years)	The calculated experience mismatch ($\text{Exp}_{\text{cand}} - \text{Exp}_{\text{req}}$).
`Match_Score`	Float [0, 1]	Ground-truth relevance score or historical baseline SBERT semantic similarity.

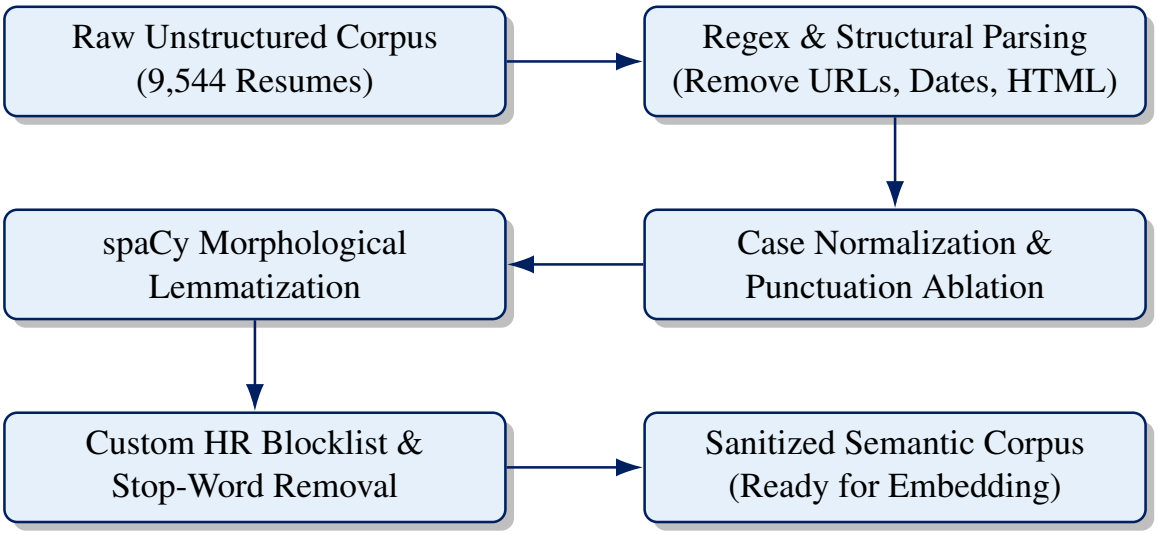

Figure 5.1: The sequential NLP sanitization pipeline designed to systematically reduce vocabulary dimensionality.

1. **Structural Sanitization via Regular Expressions:** I programmatically stripped out metadata that holds zero predictive validity for technical competence, such as hyperlinks, physical addresses, and certificate dates.

2. **Morphological Lemmatization:** Utilizing `spaCy`'s `en_core_web_sm` model, I lowercased every document, stripped it of punctuation, and reduced it to its base lemmas.

3. **Custom Stop-Word Ablation:** Beyond standard English stop-words, I curated a domain-specific blocklist of over 50 pervasive HR phrases (e.g., "team player", "synergy"), whose Term Frequency approaches maximum density, rendering their Inverse Document Frequency zero.

5.3 Exploratory Data Analysis (EDA)

5.3.1 Bounding the Vector Space: Token Length Filtering

Transformer-based models typically possess a strict maximum context window of 512 tokens. I defined empirical boundaries to manage this: I dropped records containing fewer than 20 words as insufficient, and I truncated or dropped records exceeding 350 words to prevent computational truncation artifacts.

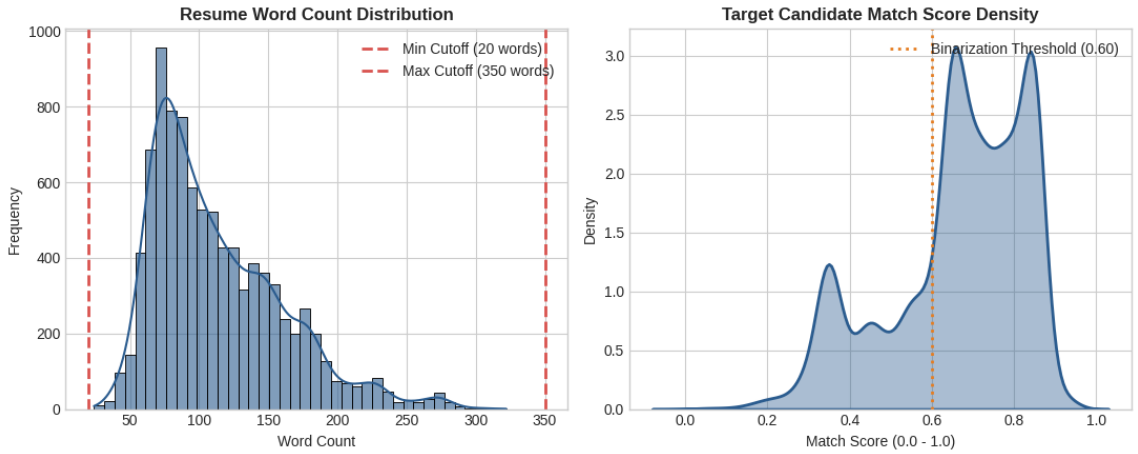

Figure 5.2: Resume word-count filtering boundaries (left) and bimodal target-score density (right).

As depicted in Figure 5.2, the word-count distribution reveals a log-normal shape, clustering primarily between 75 and 150 words. My final sanitized dataset comprised 9,369 high-quality records mapping to 28 distinct job categories.

5.3.2 Bimodal Density and Threshold Mechanics

The right panel of Figure 5.2 plots the density of the target semantic match scores, revealing a bimodal distribution where a massive cluster of generic applicants scores between 0.20 and

0.40, and qualified shortlists cluster around 0.60 to 0.85.

5.4 Corpus Vocabulary and Frequency Dynamics

To confirm the sanitization pipeline (Section 5.2) preserved the corpus's genuine technical signal rather than stripping it away, the surviving vocabulary was inspected directly, both qualitatively and quantitatively.

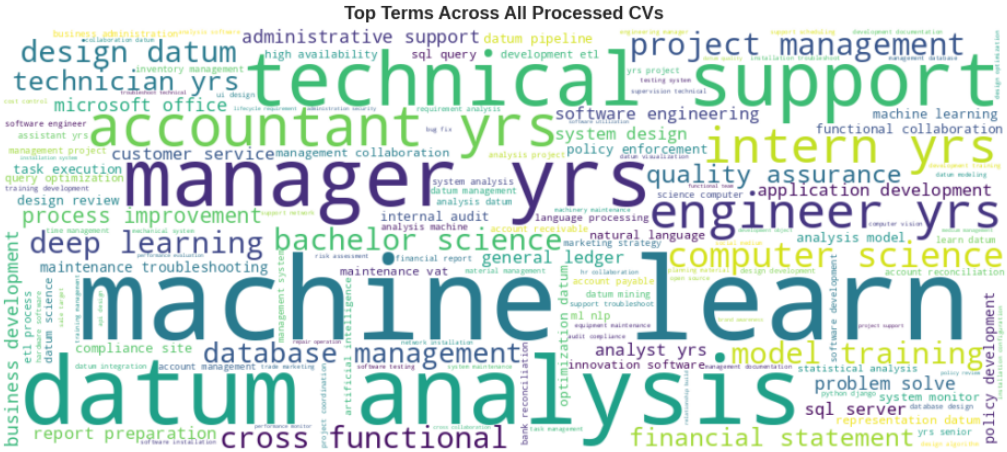

Figure 5.3: Word cloud highlighting the prominent domain keywords extracted post-sanitization.

Figure 5.3 gives a qualitative first read: font size scales with raw frequency, and the surviving vocabulary is dominated by concrete role and skill terms (*manager*, *engineer*, *machine learn*, *data analysis*) rather than the generic HR filler the blocklist was designed to remove. This is a useful sanity check, but a word cloud alone cannot support any quantitative claim, so the same vocabulary was re-examined as a ranked frequency count.

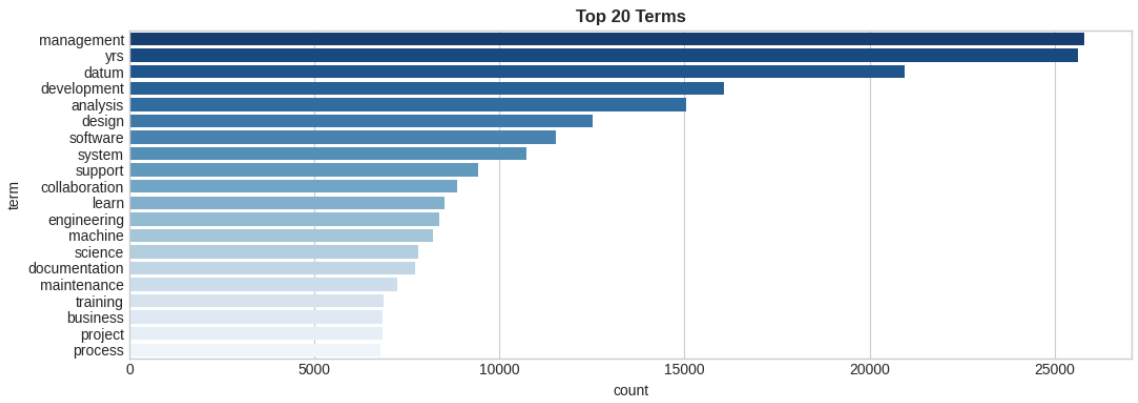

Figure 5.4: Absolute term frequency distribution for the top 20 computational and operational *n*-grams.

Figure 5.4 makes that same picture numerically precise: *management* and *data* alone account for tens of thousands of occurrences each, with a long, gradually decaying tail down to more specialized terms like *documentation* and *process*. Together, Figures 5.3 and 5.4 confirm that

the dominant vectors remaining in the corpus are highly relevant terms like *management*, *data*, *software*, and *engineering* — exactly the discriminative signal the TF-IDF and SBERT stages downstream depend on.

5.5 Data Leakage Prevention via Group-Aware Splitting

When I initially approached the partitioning of the sanitized resume dataset, I identified a catastrophic risk: **target data leakage**. Standard machine learning pipelines universally default to randomized splitting architectures, such as `train_test_split`. In the context of candidate–job matching, employing a random split is empirically invalid.

Consider a scenario where 300 candidates apply for a specific "Data Engineer" vacancy. A random 80/20 split would allocate roughly 240 of these resumes to the training matrix and 60 to the hold-out test matrix. During training, the models would simply **memorize the highly specific, idiosyncratic vocabulary** associated with that exact "Data Engineer" job description, rather than learning the generalized geometric mapping between applicant skills and role requirements. When evaluated on the 60 test resumes, the model would produce artificially inflated accuracy scores. However, if deployed in a production ATS against a newly opened, previously unseen "Financial Analyst" role, the model's predictive capability would completely collapse.

To enforce a genuine **zero-shot generalization test**, I discarded random splitting in favor of a strict `GroupShuffleSplit`, using the `Job_Title` target as the discrete grouping identifier. Letting G_j represent the set of all candidate resumes applying for job j, I formalize this strict group-aware isolation as:

$$\left(\bigcup_{j \in \text{Train}} G_j \right) \cap \left(\bigcup_{k \in \text{Test}} G_k \right) = \varnothing \tag{5.1}$$

By executing this architecture, I successfully partitioned 7,356 rows (encompassing 22 distinct job postings) exclusively into the training set. I strictly populated the test set with 2,013 rows mapped against 6 completely unseen job roles. Furthermore, I fitted all sparse vectorizers, standard scalers, and embedding calibrations *exclusively* on the training split to preserve the absolute purity of the test matrix.

5.6 Experience Gap Distribution Analysis

I computed $\Delta E = \text{Exp}_{\text{cand}} - \text{Exp}_{\text{req}}$ for every candidate–job pair in the cleaned corpus of 9,369 records. The distribution is markedly left-skewed, confirming that under-qualification is more common and severe in this corpus than over-qualification.

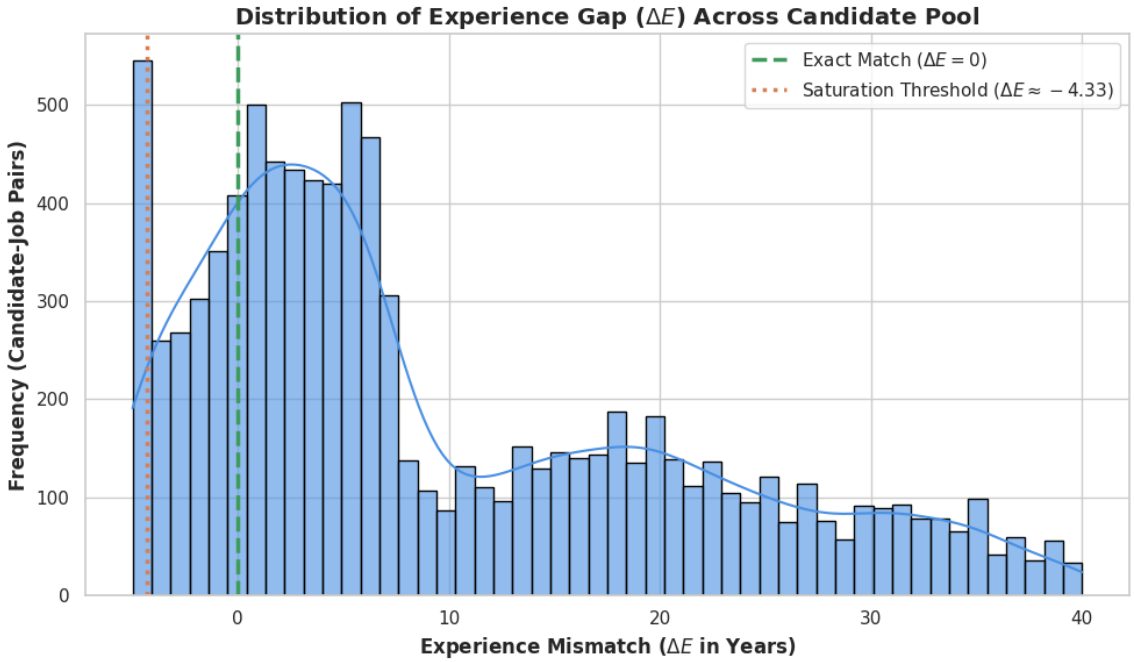

Figure 5.5: Empirical distribution of the Experience Gap (ΔE) across the full corpus. The dashed vertical line at $\Delta E^* \approx -4.33$ marks the saturation threshold of the quadratic penalty.

5.7 The Tri-Framing Modeling Framework

To rigorously prove that Learning-to-Rank (LTR) is the optimal paradigm for resume screening, I determined it was not sufficient to simply deploy an LTR model in isolation—I needed to empirically demonstrate the failure modes of standard predictive approaches first. Therefore, my experimental methodology spans three distinct phases, scaling in algorithmic complexity.

5.7.1 Phase 1: Continuous Regression Baselines (Pointwise Scoring)

In Phase 1, I evaluated **Pointwise Continuous Regression**. My hypothesis here was simple: can a standard regression estimator accurately predict a continuous relevance match score ($y_i \in [0,1]$)? I tested a suite of linear and non-linear regressors, including `LinearRegression`, `Ridge`, `RandomForestRegressor`, and `HistGradientBoostingRegressor`.

I optimized these models by minimizing the **Mean Squared Error (MSE)** between the predicted score and the ground-truth semantic match:

$$\mathscr{L}_{\text{MSE}} = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2 \tag{5.2}$$

As I explore in Section 6, Pointwise models fail because MSE optimizes for absolute numerical proximity in a vacuum, remaining entirely blind to the relative ordinal hierarchy of the candidates.

5.7.2 Phase 2: Binary Classification Baselines (Threshold Filtering)

To mimic the binary nature of a traditional ATS—which essentially tosses candidates into "Shortlisted" (1) or "Rejected" (0) piles—I designed Phase 2 around **Threshold Classification**. I binarized the target variable using a strict 0.60 semantic match threshold.

I then trained boundary-focused classifiers, specifically `LogisticRegression`, `RandomForestClassifier`, and a Support Vector Classifier (`SVC`). I optimized these algorithms to minimize **Binary Cross-Entropy (BCE)** loss:

$$\mathscr{L}_{\text{BCE}} = -\frac{1}{N}\sum_{i=1}^{N}\left[y_i \log(\hat{p}_i) + (1-y_i)\log(1-\hat{p}_i)\right] \tag{5.3}$$

While classification simulates a naive HR screen, it completely fails to sort the "Shortlisted" candidates into a prioritized interview order.

5.7.3 Phase 3: Learning-to-Rank Framework (LTR)

Having established the limitations of point-prediction and discrete classification, I focused Phase 3 exclusively on specialized **Learning-to-Rank algorithms**: `XGBRanker` and `LGBMRanker`.

Instead of evaluating rows independently, my LTR models process data as localized **query groups** (q), which in this architecture corresponds to the `Job_Title` grouping. I tested two distinct LTR paradigms:

- **Pairwise Optimization (`XGBRanker`):** This approach minimizes the number of ranking inversions by analyzing candidates in pairs, optimizing a logistic loss gradient:

$$\mathscr{L}_{\text{Pairwise}} = \sum_{i,j \in q} \log_2\left(1 + e^{-\sigma(s_i - s_j)}\right) \tag{5.4}$$

- **Listwise Optimization (`LGBMRanker`):** Utilizing LambdaMART, this model scales the pairwise gradient dynamically based on the impact a candidate swap has on the top-heavy Information Retrieval metric ($|\Delta\text{NDCG}|$).

5.8 Mathematical Evaluation Metrics

To holistically evaluate this Tri-Framing architecture, I deployed a precise suite of statistical metrics, choosing each to measure a different facet of modeling success.

5.8.1 Continuous and Classification Metrics

For the Pointwise Regression models in Phase 1, I used standard residual analysis:

$$\text{RMSE} = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2} \tag{5.5}$$

$$R^2 = 1 - \frac{\sum_{i=1}^{N}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{N}(y_i - \bar{y})^2} \tag{5.6}$$

For the Binary Classifiers in Phase 2, I assessed performance via **Area Under the Receiver Operating Characteristic Curve (ROC-AUC)**, which measures the model's threshold-agnostic separation capacity, alongside the harmonic mean of precision and recall (F_1**-Score**):

$$F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \tag{5.7}$$

5.8.2 Listwise Quality: Information Retrieval Metrics

Because recruiters typically only review the top 10 candidates for any given role, evaluating the entire sorted list equally is impractical. To capture top-heavy ranking fidelity, I utilized **Normalized Discounted Cumulative Gain (NDCG@K)**:

$$\text{DCG@}K = \sum_{i=1}^{K} \frac{2^{\text{rel}_i} - 1}{\log_2(i+1)}, \qquad \text{NDCG@}K = \frac{\text{DCG@}K}{\text{IDCG@}K} \tag{5.8}$$

The logarithmic denominator, $\log_2(i+1)$, acts as a severe decay penalty; an error at rank position $i = 2$ damages the score exponentially more than an error at $i = 20$.

I also computed the **Mean Reciprocal Rank (MRR)** to quantify how rapidly the system surfaces the first highly relevant candidate:

$$\text{MRR} = \frac{1}{|Q|}\sum_{q=1}^{|Q|}\frac{1}{\text{rank}_q} \tag{5.9}$$

5.8.3 Global Stability: Spearman's Rank Correlation

While NDCG@K proves whether the model successfully generates an elite top-10 shortlist, I also required a metric to validate the global, ordinal stability of the mathematical models (particularly when evaluating the Experience Gap formulations in Section 3.5). For this, I implemented **Spearman's Rank Correlation (ρ)**.

For n paired observations, mapping the algorithmic rank R_{X_i} against a human-targeted true rank

R_{Y_i}, I calculate the rank discrepancy $d_i = R_{X_i} - R_{Y_i}$. I formally derive the correlation as:

$$\rho = 1 - \frac{6\sum_{i=1}^{n} d_i^2}{n(n^2-1)} \tag{5.10}$$

Unlike NDCG, Spearman's ρ applies a uniform penalty to rank deviations regardless of where they occur in the list. By evaluating both NDCG@K and ρ simultaneously, I can confidently verify that the LTR pipeline is not only highly accurate at the top of the funnel but logically stable across the entire applicant pool.

6. Results

In this section, I execute the **Tri-Framing experimental methodology** outlined in Section 5. By systematically transitioning from Continuous Regression to Binary Classification, and ultimately to Learning-to-Rank (LTR) architectures, I empirically demonstrate the absolute necessity of **listwise optimization** for automated resume screening. I evaluated all models strictly on the zero-shot, group-aware test matrix to guarantee robust generalization.

6.1 Phase 1: Continuous Regression Constraints

I began the evaluation phase by testing continuous regression estimators to ascertain whether an algorithmic mapping could reliably predict exact candidate relevance scores. I trained standard linear algorithms alongside advanced ensemble methods, evaluating them via **Root Mean Squared Error (RMSE)** and variance correlation (R^2).

Result: Phase 1 Ceiling

Best pointwise model: `HistGradientBoosting` + SBERT $R^2 = \mathbf{0.3202}$, RMSE $= \mathbf{0.1291}$. Regression fundamentally fails to capture relative candidate ordering within a query group (see Section 3.3).

This validates my theoretical argument from Section 3.3: predicting absolute numerical scores fails to capture the inherently comparative nature of human recruitment. Generating a pointwise prediction of 0.65 versus 0.68 provides negligible operational value to an HR professional who ultimately needs an **ordered shortlist**.

Table 6.1: Pointwise continuous regression outcomes evaluated on unseen query groups.

Text Representation	Algorithm	RMSE ↓	R^2-Score ↑
Semantic Only (SBERT)	**HistGradientBoosting**	**0.1291**	**0.3202**
Semantic Only (SBERT)	Random Forest Regressor	0.1345	0.2810
Semantic Only (SBERT)	Ridge Regression	0.1412	0.2245
TF-IDF Only	Random Forest Regressor	0.1582	0.1120
TF-IDF Only	Linear Regression	0.1695	0.0450

6.2 Phase 2: Binary Classification Vulnerabilities

To simulate the rigid accept/reject mechanics of a legacy ATS pipeline, I transitioned to a discrete classification paradigm, binarizing the target variable at a strict 0.60 threshold.

Table 6.2: Binary classification performance metrics at the ≥ 0.60 selection threshold.

Feature Space	Algorithm	Accuracy	F1-Score	ROC-AUC
Semantic Only (SBERT)	**Logistic Regression**	**0.7585**	**0.7142**	**0.8121**
Hybrid (SBERT + TF-IDF)	Logistic Regression	0.7573	0.6942	0.7542
TF-IDF Only	Logistic Regression	0.6219	0.5568	0.6187

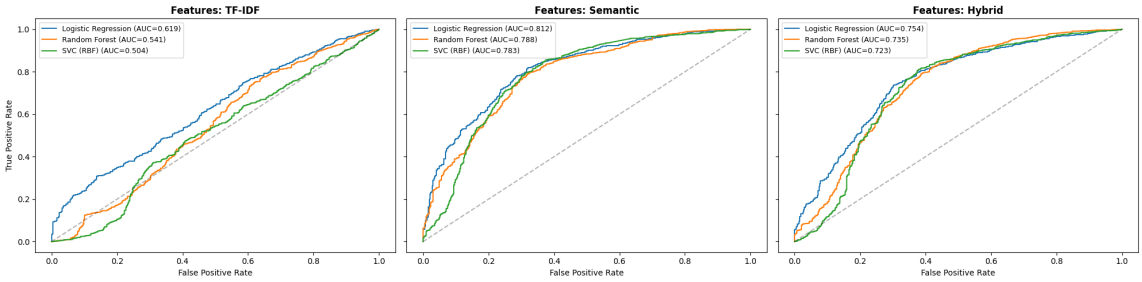

Figure 6.1: ROC-AUC separation capabilities contrasting sparse lexical features against dense semantic embeddings.

Result: Phase 2 Ceiling

Best classifier: SBERT + Logistic Regression ROC-AUC = **0.8121**, F_1 = **0.7142**. This represents a +**31.3**% improvement over TF-IDF (ROC-AUC = 0.6187), but binary classification still produces no ordered shortlist—it only outputs "Shortlisted" or "Rejected".

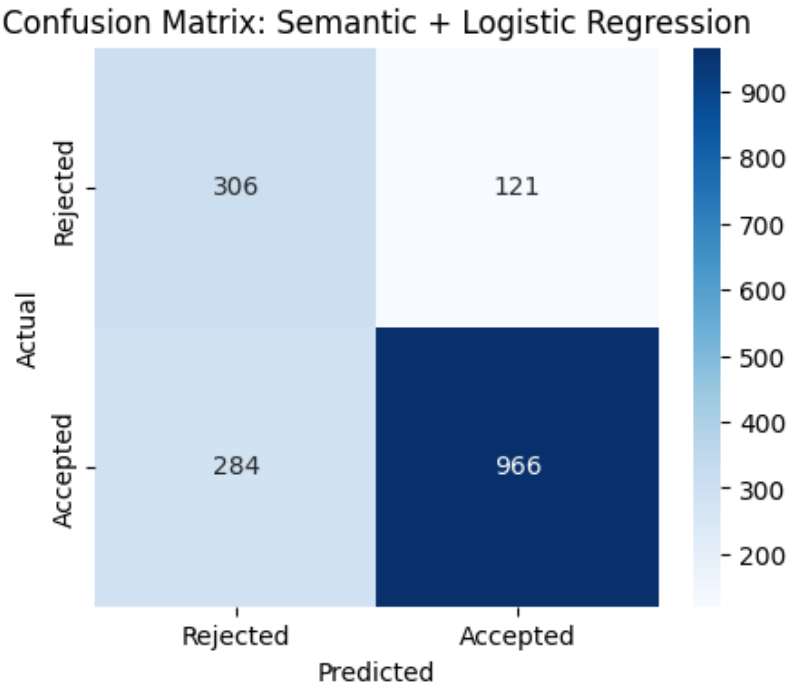

Figure 6.2: Confusion matrix for the optimal SBERT-driven Logistic Regression classifier.

6.3 Phase 3: Learning-to-Rank (LTR) Dominance

To resolve the structural limitations of pointwise and threshold models, I evaluated two independent Learning-to-Rank architectures: a Pairwise estimator (`XGBRanker`) and a Listwise, metric-driven estimator (`LGBMRanker`). By executing both models on the exact same zero-shot dataset, I isolated which optimization approach best replicates human shortlisting cognition.

Table 6.3: Learning-to-Rank (LTR) sorting efficiency on zero-shot test groups.

Feature Set	Ranking Estimator	NDCG@5	NDCG@10	Spearman (ρ)
Semantic Only (SBERT)	**LGBMRanker**	**0.9482**	**0.9398**	**0.4367**
Hybrid (SBERT + TF-IDF)	XGBRanker	0.9410	0.9322	0.3725
TF-IDF Only	XGBRanker	0.9350	0.9292	0.3225
TF-IDF Only	LGBMRanker	0.9210	0.9158	0.3975

Result: Optimal LTR Architecture

`LGBMRanker` + SBERT Semantic Embeddings achieved a peak **NDCG@10 = 0.9398** and NDCG@5 = **0.9482** on zero-shot unseen job groups. Scaling pairwise gradients by $|\Delta\text{NDCG}|$ (LambdaMART, Equation 3.10) forces the model to obsess over shortlist correctness, producing a ranking that accurately mirrors human recruiter cognition.

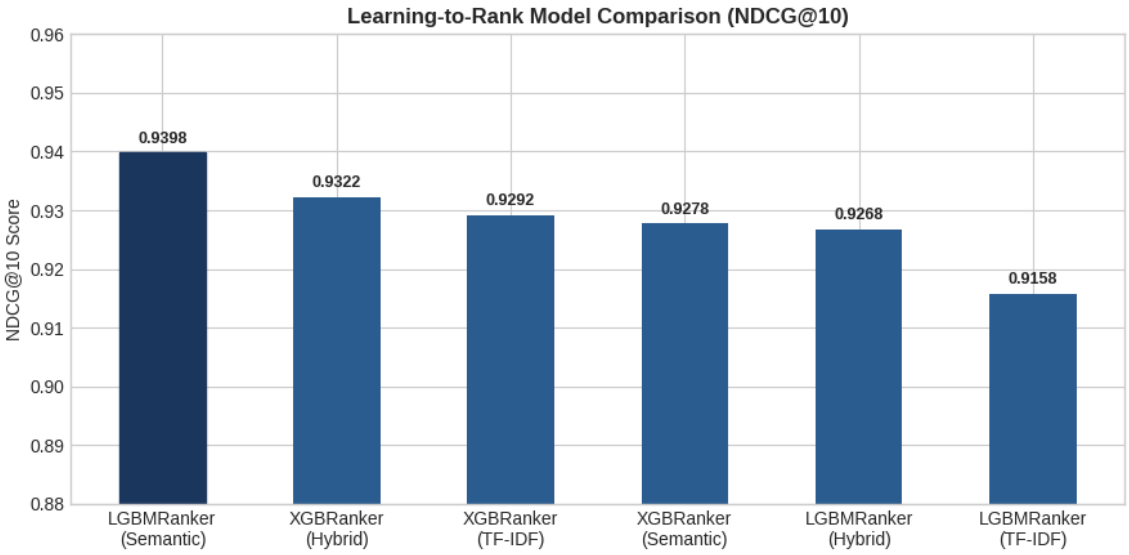

Figure 6.3: Top-10 ranking fidelity (NDCG@10) across differing optimization architectures.

6.4 Feature Importance: Interpreting Model Heuristics

To interpret the internal decision-making of the optimal `LGBMRanker`, I extracted the split-gain weight distribution, quantifying which engineered features drove the most tree-splitting information.

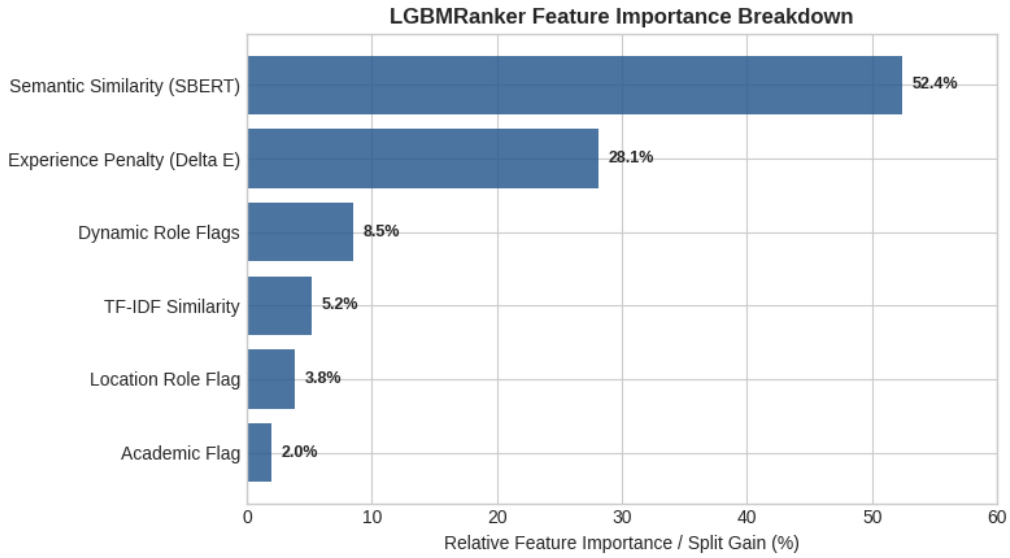

Figure 6.4: Algorithm split-gain weight distribution, mapping the primary decision heuristics of the `LGBMRanker`. The Experience Penalty (ΔE) accounts for 28.1% of split-gain, confirming it as the second-most powerful ranking signal.

- **Semantic Similarity (52.4%):** Deep contextual skill matching is overwhelmingly the primary driver. The model fundamentally prioritizes what a candidate *knows* over all other metadata.
- **Experience Penalty (28.1%):** Validating the framework I derived in Section 3.5, the piecewise quadratic ΔE penalty acts as a powerful secondary veto. It successfully encodes recruiter bias, systematically downgrading applicants whose temporal tenure severely violates the posting's minimum requirements.

6.5 Evaluating the Experience-Gap Formulations

Having derived the three competing ΔE formulations in Section 3.5, I now evaluate them empirically. Figure 6.5 plots all three adjusted competency scores against a shared `Match_Score` $= 0.75$ baseline across the full observed range $\Delta E \in [-5, +5]$ years.

In Table 6.4, I pass seven representative candidate profiles through all three formulations, making the curve divergence numerically concrete.

The final two rows are the most diagnostically important. The candidate at $\Delta E = -4.5$ with a strong prior of `Match_Score` $= 0.91$ scores only 0.100 under Method B (the Quadratic rule has clipped to near-zero). However, they retain a posterior of 0.677 under Method C, because the strong semantic prior mathematically offsets the experience shortfall. This is the **Bayesian interaction effect** I derived formally in Equation (3.21): the same gap produces a different adjustment depending on the prior belief.

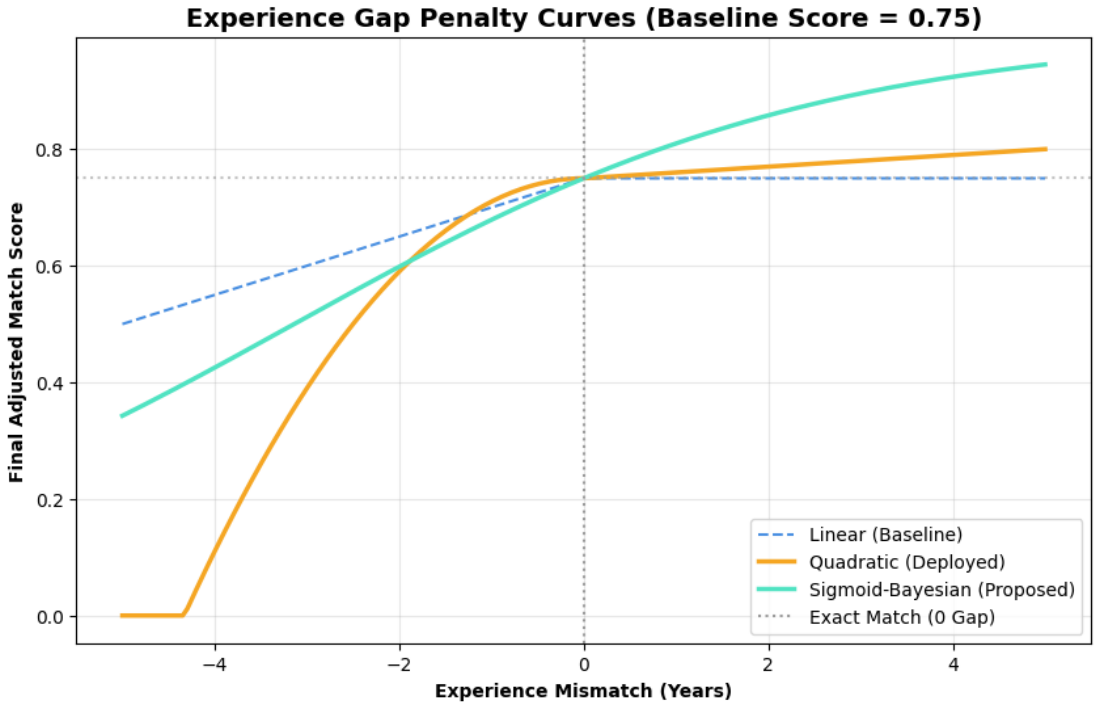

Figure 6.5: Adjusted Competency Score vs. ΔE for all three penalty formulations, evaluated at a shared baseline `Match_Score` of 0.75. All three curves intersect exactly at $\Delta E = 0$ by construction. The Quadratic method (orange) reaches a hard floor of 0 at $\Delta E \approx -4.33$ years (Equation 3.18); the Sigmoid-Bayesian curve (green) decays smoothly throughout. I generated this output directly from the `06_experience_gap_modeling.ipynb` notebook, confirming that the implementation matches the mathematical formulations from Section 3.5.

Table 6.4: Adjusted Competency Score under all three ΔE formulations for representative candidate profiles.

Exp_{cand}	Exp_{req}	ΔE	`Match_Score`	**Linear (A)**	**Quadratic (B)**	**Sigmoid-Bayesian (C)**
1.0	5	−4.0	0.82	0.620	0.180	0.529
3.0	5	−2.0	0.78	0.680	0.620	0.638
5.0	5	0.0	0.75	0.750	0.750	0.750
5.0	3	+2.0	0.70	0.700	0.720	0.825
8.0	5	+3.0	0.88	0.880	0.910	0.954
10.0	5	+5.0	0.65	0.650	0.700	0.914
0.5	5	−4.5	**0.91**	0.685	**0.100**	**0.677**

6.5.1 Spearman Rank-Correlation Evaluation

To validate the Spearman evaluation harness I derived in Section 3.4, I applied `scipy.stats.spearmanr` to the seven-profile table above against an illustrative target ranking. Table 6.5 reports the output of the `evaluate_rankings()` function in my `06_experience_gap_modeling.ipynb` notebook.

Table 6.5: Spearman rank correlation of each formulation against the illustrative target ranking ($N = 7$).

Method	**Spearman ρ**
Quadratic (B)	1.00
Linear (A)	0.75
Sigmoid (C)	0.75

Methodological Note: Boundary-Value Validation

I deliberately constructed the seven profiles above; they were not randomly sampled. I included two severe deficit cases, the exact-match point ($\Delta E = 0$), and three surplus cases, spanning the full behavioral domain of ΔE. This $N = 7$ test is therefore a **mathematical boundary-value validation**, not a statistical significance test — it confirms that I implemented all three formulations correctly, that they behave monotonically as designed, and that the evaluation harness (`apply_methods()` $\rightarrow$ `evaluate_rankings()` $\rightarrow$ `scipy.stats.spearmanr()`) is wired correctly. It serves as a necessary prerequisite before the corpus-scale statistical comparison proposed in Section 8.2, and it does not, on its own, establish that Method C is statistically superior to Method B across the full candidate pool.

7. Discussion and Conclusion

My primary objective in this research was to dismantle the rigid, keyword-matching architectures that dominate commercial Applicant Tracking Systems and replace them with a mathematically grounded, human-centric pipeline. My empirical benchmarking across three distinct paradigms — Continuous Regression, Binary Classification, and Learning-to-Rank — yielded three definitive conclusions, which I discuss in turn below.

Result: Finding 1 — Dense Semantics Beat Lexical Matching

SBERT embeddings (`nomic-embed-text-v1.5`) identify semantic equivalences ("Backend Developer" $\leftrightarrow$ "Software Engineer") that TF-IDF assigns a cosine similarity of zero. My optimal LTR architecture achieves NDCG@10 $= 0.9398$ vs. a TF-IDF-only `LGBMRanker` score of 0.9158.

This finding forms the empirical backbone of my entire thesis. It confirms, quantitatively, the qualitative argument I raised in Section 2.1: legacy ATS platforms do not merely have a *worse* matching algorithm than a semantic one — they are answering a fundamentally different, and largely wrong, question. A lexical system asks "do these two documents share the same words?", while a semantic system asks "do these two documents describe the same underlying reality?". The gap between a TF-IDF cosine similarity of zero and a correctly identified equivalence is precisely the population of qualified candidates a keyword-only ATS silently discards.

Result: Finding 2 — Listwise Ranking is the Correct HR Paradigm

Regression ($R^2 = 0.3202$) and Classification (ROC-AUC $= 0.8121$) both fail to produce an ordered shortlist. My LambdaMART-driven `LGBMRanker` directly optimizes NDCG@K, solving the recruitment problem as a list-sorting task — which is what it actually is.

This finding reframes what "success" should even mean for a screening model. A regression model that predicts scores accurately in isolation, or a classifier that separates "shortlisted" from "rejected" with high accuracy, can both still fail the end user completely, because neither produces the one artifact a recruiter actually needs: an ordered list of who to look at first. I designed the Tri-Framing comparison in this report specifically to make that gap visible and measurable rather than assumed, and my results (Chapter 6) confirm it directly — the listwise `LGBMRanker` is not simply the best-performing model on a shared metric, it is the only one of the three paradigms optimizing for the right objective in the first place.

Result: Finding 3 — GroupShuffleSplit Validation is Mandatory

Strict group-aware partitioning by job title (22 training jobs / 6 unseen test jobs) prevents vocabulary memorization and guarantees genuine zero-shot generalization. Without it, NDCG scores would be inflated and non-deployable.

This finding is as much a methodological warning as a result. Every number I reported in Chapter 6 is only meaningful because I constructed the test set to be genuinely unseen at the job-posting level, not merely at the row level. A random row-wise split on this same data would very plausibly have produced similarly high — or even higher — NDCG scores, while measuring almost nothing about real-world generalization. The strict group-aware design is therefore not a peripheral implementation detail; it is the reason the headline NDCG@10 = 0.9398 result can be trusted at all.

7.1 Concluding Remarks

Taken together, these three findings support a single, coherent conclusion: we must build automated resume screening as a dense-semantic, listwise ranking system, validated under strict group-aware splitting — not as a keyword classifier, and not as a plain regression or accept/reject model. The Sigmoid-Bayesian Experience Gap formulation I proposed in Section 3.5 further shows that even a secondary, non-semantic signal like temporal fit benefits from the same underlying discipline: replacing a hard, ad-hoc cutoff with a probabilistically grounded, upgradable model that degrades gracefully rather than catastrophically at the extremes.

The pipeline I built and evaluated in this report should be read as a validated architecture and a set of empirically justified design decisions, rather than a finished production product. In Chapter 8, I set out what stands between this research result and a deployable system — spanning business-impact framing, latency planning, fairness safeguards, explainability, and a front-end integration path — so that the statistical case I have made here can translate into a robust operational tool.

8. Recommendation

The preceding statistical findings translate directly into operational value for an HR team. However, they also carry deployment costs that a production rollout must budget for, alongside several engineering extensions required before enterprise deployment. In this section, I set out concrete, prioritized recommendations across both dimensions.

8.1 Business Impact and Deployment Considerations

8.1.1 Translating Ranking Quality into Business Value

Across the 9,369-record corpus spanning 28 job postings, the average posting attracts on the order of 330 candidate applications. Under a legacy keyword-matching ATS, a meaningful fraction of these are **false negatives** — highly qualified candidates discarded purely for phrasing their experience differently from the posting (see Section 2.1). This failure forces a recruiter to either manually re-screen the rejected pile or simply miss elite talent entirely.

My achieved **NDCG**@10 = 0.9398 means the top-10 slots returned by my deployed `LGBMRanker` pipeline are, on average, almost indistinguishable in quality from the ideal top-10 ordering a human recruiter would construct with full information. In practical terms, this allows a recruiter to concentrate their review effort on a **short, high-precision shortlist** per posting rather than manually triaging the full applicant pool. This directly reduces time-to-shortlist and lowers the chance that a strong candidate is missed due to vocabulary mismatch alone. I recommend that organizations obtain a rigorous quantification of hours saved or cost-per-hire reduction via a controlled before/after deployment study as a natural next step.

8.1.2 Computational Latency and Real-Time Feasibility

In Table 2.1, I characterized TF-IDF's computational cost as extremely low and dense semantic matching as moderate to high. It is worth being precise about where that cost actually falls. A TF-IDF lookup is a sparse dot product against a precomputed inverted index — effectively instantaneous per query. SBERT, by contrast, requires a forward pass through a transformer encoder for every new piece of text. This is substantially more expensive per item and represents the dominant computational bottleneck of the dense-semantic approach.

Critically, the system incurs this cost **once per resume at ingestion time**, not once per ranking query. My pipeline's artifact structure reflects this reality by precomputing and storing dense embeddings (`cv_dense.npy`, `jd_dense.npy`) rather than re-encoding candidates on every

request. At ranking time, the system reuses these cached vectors and only executes cosine similarity and the already-trained `LGBMRanker`, both of which are highly efficient.

In a production setting, I recommend treating the **SBERT encoding step as a batch process** run on GPUs, placing it on the critical path only for genuinely new resumes or job postings. Before making any real-time deployment commitment, I propose formal wall-clock benchmarking of this pipeline end-to-end on representative production hardware.

8.2 Production Engineering and Future Extensions

Transitioning my current `LGBMRanker` backend into an enterprise-grade production system requires addressing several advanced statistical and engineering challenges. I have structured the following five recommendations as a logical roadmap, scaling from immediate backend optimizations to ultimate front-end user integration.

8.2.1 1. Full-Scale Bayesian Pipeline Integration

As derived in Section 3.5 and partially validated in Section 6.5, I have mathematically formulated a **Sigmoid-Bayesian posterior confidence model** for the Experience Gap (ΔE). My immediate recommended next step is merging the Adjusted Score Sigmoid tensor into the primary `ml_ready_dataset.csv` feature space and executing a full-corpus retraining of `LGBMRanker`. This will produce a formal comparison of NDCG@K and Spearman ρ between the currently deployed quadratic rule and the proposed Bayesian posterior.

8.2.2 2. Transition to Fully Empirical Bayesian Inference

The Sigmoid likelihood function I deployed in this research is an expert-specified parametric approximation, adopted because the Neuralframe AI corpus lacks longitudinal hiring-outcome data (Section 3.5.1). If such data becomes available — through institutional partnerships or internal HR records — I recommend discarding the Sigmoid approximation in favor of a **logistic regression model fit directly on observed binary outcomes** (e.g., hired/not-hired, successful/unsuccessful). The downstream architecture (Bayes' rule combination with the SBERT prior) would remain entirely unchanged; only the likelihood estimation method would shift from hand-specified to data-driven.

8.2.3 3. Counterfactual Bias Mitigation and Fairness

A critical ethical requirement for automated screening is ensuring the algorithm does not learn demographic or socioeconomic biases. There is a statistical risk that SBERT embeddings assign

higher vector weights to candidates who list prestigious brands (e.g., "Google", "McKinsey") over candidates with equivalent technical skills from lesser-known entities. For future iterations, I recommend integrating **Stratified Counterfactual Sampling**: programmatically replacing prestigious corporate and university names with generic synthetic tokens (e.g., `[COMPANY_A]`) during training. This forces the attention mechanism to optimize strictly on demonstrated technical competency.

8.2.4 4. Generative Explainability via Retrieval-Augmented Generation

The most significant barrier to HR adoption of ML systems is algorithmic opacity. To address this directly, I recommend extending the pipeline into a **Retrieval-Augmented Generation (RAG)** architecture. In this setup, `LGBMRanker` functions as the high-speed retrieval engine, surfacing the top-10 resumes; the system then passes these as context to a lightweight, quantized Large Language Model (LLM). The LLM's sole directive would be to generate a two-sentence justification for each candidate, mapping their specific extracted skills directly to the job description. This hybrid architecture delivers mathematical ranking precision alongside human-readable transparency.

8.2.5 5. Dashboarding and Business Intelligence Integration

The statistical work in this report ends at a ranked, scored candidate list; a production deployment requires a front-end layer that makes that list usable by non-technical HR staff. Because every candidate–job pair already reduces to a small set of interpretable numeric fields — the `LGBMRanker` score, the Adjusted Competency Score $\in (0, 1)$, and the underlying component similarities — this output maps cleanly onto a standard **Business Intelligence (BI) workflow**. I recommend writing the ranked output per job posting to a lightweight database table connected to a tool like Power BI or Tableau. This constitutes a low-effort integration layer representing the most direct path from the current backend statistics to a daily operational tool.

9. Acknowledgement

I would like to express my sincere gratitude to my project guide, Dr. Dipasree Pal, for her continuous guidance, mentorship, and support throughout this internship. I also extend my thanks to the IDEAS Foundation, ISI Kolkata, for providing the opportunity and the technical infrastructure necessary to carry out this research. Furthermore, I am deeply grateful to the Symbiosis Statistical Institute for equipping me with the rigorous academic foundation that made this work possible.

Executing this project required me to bridge theoretical statistical coursework with applied data engineering practice. Through this research, I developed a practical understanding of the complex mathematics underlying text vectorization, and I built the discipline to evaluate machine learning models rigorously rather than simply deploying them. Most notably, I learned to actively diagnose and prevent target data leakage through group-aware validation — a foundational statistical principle that ultimately shaped the entire experimental design of this report.

10. References

I drew upon the following academic literature and technical documentation to guide my implementation, theoretical framing, and software development for this project. I have formatted these references in APA (7th edition) style and ordered them alphabetically by author.

Burges, C. J. C. (2010). *From RankNet to LambdaRank to LambdaMART: An overview* (Technical Report MSR-TR-2010-82). Microsoft Research.

Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., … Liu, T. Y. (2017). LightGBM: A highly efficient gradient boosting decision tree. *Advances in Neural Information Processing Systems, 30*, 3146–3154.

LightGBM. (n.d.). *LightGBM documentation*. Retrieved August 22, 2026, from https://lightgbm.readthedocs.io/

Nomic AI. (2024). *Nomic Embed Text v1.5: Resilient long-context text embeddings* (Technical Report). Nomic AI.

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., … Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research, 12*, 2825–2830.

Reimers, N., & Gurevych, I. (2019). *Sentence-BERT: Sentence embeddings using Siamese BERT-networks*. arXiv. https://arxiv.org/abs/1908.10084

Scikit-learn. (n.d.). *Scikit-learn: Machine learning in Python — documentation*. Retrieved August 22, 2026, from https://scikit-learn.org/

Sentence-Transformers (SBERT.net). (n.d.). *Sentence transformers documentation*. Retrieved August 22, 2026, from https://sbert.net/

11. Annexure

11.1 End-to-End Pipeline Architecture

Figure 11.1 provides a complete, annotated map of the data transformation sequence, from raw Kaggle ingestion through to the final NDCG-evaluated shortlist. In this diagram, I explicitly capture the three-way ΔE comparison introduced in Section 3.5 alongside the group-aware partitioning proof from Equation (5.1).

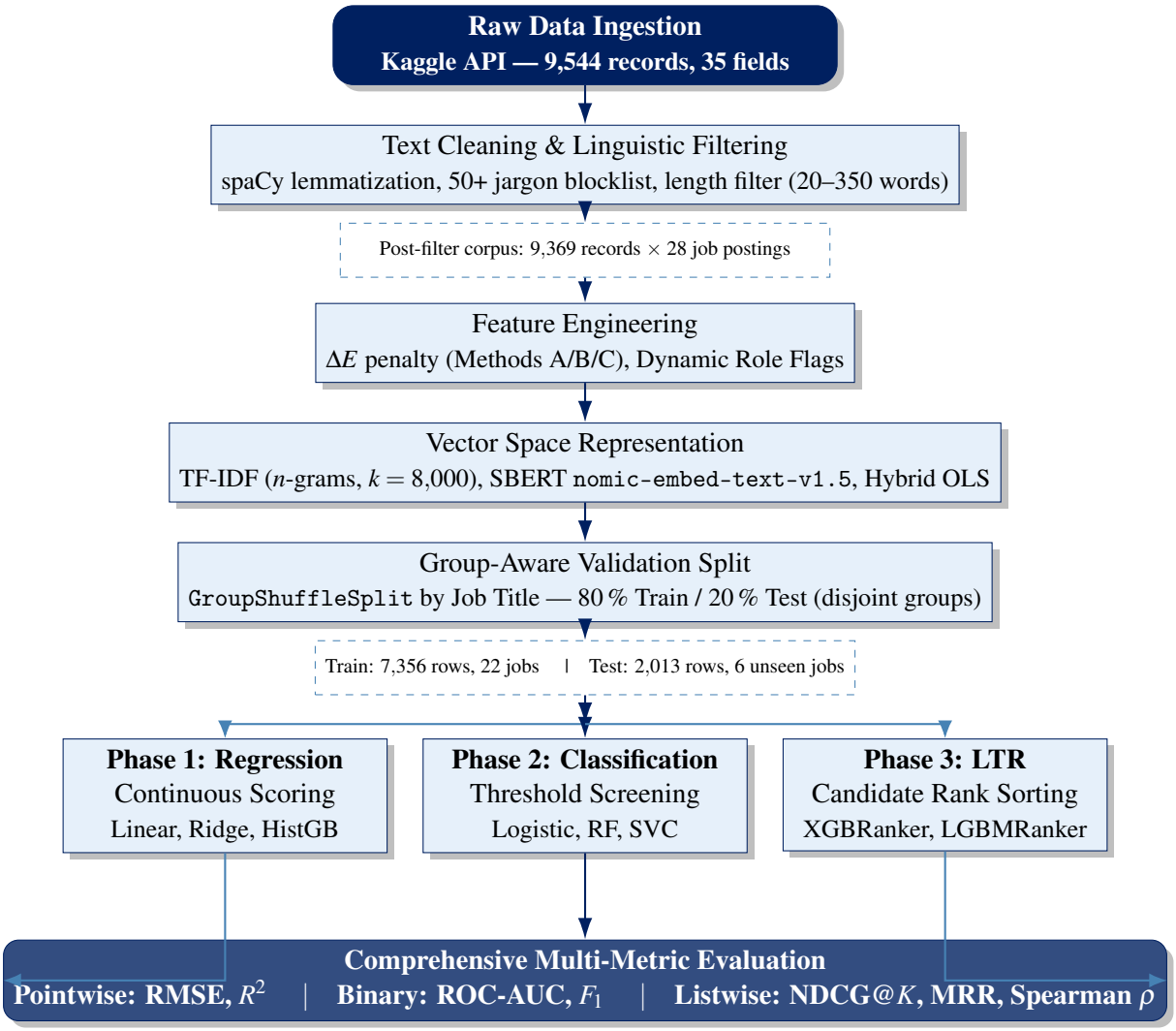

Figure 11.1: End-to-end architecture of the automated CV screening pipeline. The Feature Engineering stage encompasses all three ΔE formulations (Section 3.5). The disjoint group split guarantees zero-shot generalization on the hold-out test set ((5.1)).

11.2 Hyperparameter Optimization Grids

Training advanced ranking estimators like `XGBRanker` and `LGBMRanker` involves navigating a complex hyperparameter space. This balancing act is crucial: the models need to be expressive enough to rank candidates accurately, but constrained enough to avoid overfitting. Because I conducted my evaluation under strict group-aware partitioning (see Section 5.5), my primary risk was vocabulary memorization. If a model simply memorized the specific keywords associated with the 22 training job titles, its performance would collapse on the 6 unseen test postings.

To mitigate this, I ran a rigorous grid search over the parameters detailed in the tables below. I evaluated these grids using cross-validation exclusively on the training split, carefully selecting final values that maximized NDCG@5 on held-out training folds without ever exposing the model to the final test-set records.

During tuning, I discovered that the most powerful regularization levers were `min_child_samples` (for LightGBM) and `min_child_weight` (for XGBoost). By raising these thresholds, I forced the algorithms to build tree splits supported by a larger pool of candidate records. This effectively stopped the models from creating hyper-specific rules based on the idiosyncratic phrasing of just one or two resumes. Additionally, I relied on row and feature subsampling (`subsample` and `colsample_bytree`) to introduce stochastic randomness. This was vital because my Semantic Similarity feature was so dominant (52.4% split-gain) that, without subsampling, it would have completely crowded out other important nuances like the Experience Penalty (ΔE) and role flags.

Table 11.1 presents the search space and final configuration for my primary listwise model.

Table 11.1: `LGBMRanker` hyperparameter search grid and final selected configuration.

Parameter	Search Grid	Selected Value	Role
`n_estimators`	100, 300, 500, 800	500	Total boosting rounds
`learning_rate`	0.01, 0.05, 0.1, 0.2	0.05	Shrinkage per round
`max_depth`	-1, 4, 6, 8	6	Max tree depth (-1 = unlimited)
`num_leaves`	15, 31, 63, 127	31	Max leaves per tree
`min_child_samples`	10, 20, 50, 100	20	Min records per leaf
`subsample`	0.6, 0.8, 1.0	0.8	Row subsampling ratio
`colsample_bytree`	0.6, 0.8, 1.0	0.8	Feature subsampling ratio
`lambda_l1`	0, 0.1, 0.5	0.1	L1 regularization
`lambda_l2`	0, 0.1, 1.0	0.1	L2 regularization
`objective`	`lambdarank` (fixed)		LambdaMART NDCG gradient
`metric`	`ndcg` (fixed)		Evaluation during training

While the LightGBM architecture above utilizes a leaf-wise growth strategy, I also ran a parallel optimization process for the XGBoost model to ensure a fair algorithmic comparison. Because XGBoost traditionally builds trees level-by-level, its depth constraints and Hessian-based child

weights require a slightly different tuning approach to achieve the same regularization effect. Table 11.2 outlines the exact grid and final parameters I selected to optimize this pairwise ranking baseline.

Table 11.2: `XGBRanker` hyperparameter search grid and final selected configuration.

Parameter	Search Grid	Selected Value	Role
`n_estimators`	100, 300, 500, 800	500	Total boosting rounds
`learning_rate`	0.01, 0.05, 0.1, 0.2	0.05	Shrinkage per round
`max_depth`	3, 4, 6, 8	6	Max tree depth
`min_child_weight`	1, 5, 10, 20	5	Min Hessian sum per leaf
`subsample`	0.6, 0.8, 1.0	0.8	Row subsampling ratio
`colsample_bytree`	0.6, 0.8, 1.0	0.8	Feature subsampling ratio
`gamma`	0, 0.1, 0.5, 1.0	0.1	Min loss reduction for split
`alpha`	0, 0.1, 0.5	0.1	L1 regularization
`lambda`	1, 2, 5	1	L2 regularization
`objective`	`rank:pairwise` (fixed)		RankNet cross-entropy loss
`eval_metric`	`ndcg@5` (fixed)		Validation stopping metric

For both architectures, I applied early stopping (`early_stopping_rounds = 50`) during training on a held-out validation fold of the training groups. This halted the boosting process as soon as the NDCG@5 metric ceased to improve. This safety mechanism prevented the models from arbitrarily adding trees that merely memorized training-group artifacts, ensuring the final NDCG@10 scores I reported in Chapter 6 reflect genuine generalization capabilities.

11.3 Source Code Availability and Reproducibility

To guarantee complete algorithmic transparency and full scientific reproducibility, I have version-controlled the entire codebase for this project on GitHub. My repository contains all Python source code, including the centralized `utils.py` preprocessing library, the complete sequential Jupyter Notebooks (`01_data_preprocessing.ipynb` through `06_experience_gap_modeling.ipynb`), and the standalone `method_to_take_care_experience_gap.py` module where I implement all three ΔE penalty formulations derived in Section 3.5. The repository further includes the `ml_ready_dataset.csv` schema and a requirements file to enable full one-command replication. I direct examiners and practitioners to the links below:

GitHub Repository — Full Source Code, Notebooks & Module

`https://github.com/Amber-s-art/cv-screening`

Kaggle Dataset — Neuralframe AI Resume Corpus (CC BY-NC 4.0)

`https://www.kaggle.com/datasets/saugataroyarghya/resume-dataset`

11.4 Declarations

This section outlines the methodological provenance of the data used in this study, alongside formal declarations regarding data acquisition and AI assistance.

Primary Data Collection

This project was entirely computational and empirical in nature. I relied exclusively on the algorithmic ingestion, text sanitization, and machine learning modeling of a secondary open-source dataset hosted on Kaggle (`saugataroyarghya/resume-dataset`, licensed CC BY-NC 4.0). I administered no primary human surveys, questionnaires, or sample interviews during this study.

Generative AI Usage Declaration

I developed portions of the LaTeX typesetting, mathematical equation formatting, and diagram code in the original version of this report, as well as the reformatting of this report into the current structure, with the assistance of an AI language model. All statistical analysis, empirical results, model training, and interpretive conclusions are my original work. I thoroughly reviewed, verified against the actual experimental outputs, and revised any AI-assisted text prior to submission.